



CAHIER DES CHARGES

Zümm

Système de gestion et de suivi apicole

Projet Application de gestion apicole (mini-projet J2EE)
Type Cahier des charges fonctionnel et technique
Version 1.0
Date 13 juillet 2026
Méthode Manager-GO - Élaboration d'un CdC

Ruche connectée & tableaux de bord de performance

Document conforme au plan type : Contexte / Objectifs / Périmètre /
Description fonctionnelle des besoins / Enveloppe budgétaire / Délais

Table des matières

Historique des versions	5
1 Contexte et définition du problème	6
1.1 Présentation générale	6
1.2 Problématique	6
1.3 Finalité	6
2 Objectifs du projet	7
2.1 Objectif principal	7
2.2 Objectifs spécifiques	7
2.3 Résultats attendus (indicateurs de réussite)	7
3 Périmètre du projet	8
3.1 Domaine couvert	8
3.2 Modèle métier et règles de gestion	8
3.3 Utilisateurs et droits d'accès	9
3.4 Hors périmètre	9
4 Description fonctionnelle des besoins	10
4.1 Gestion des entités (CRUD)	10
4.2 Planification et suivi des visites	10
4.2.1 Rapport de visite	10
4.3 Tableaux de bord	10
4.3.1 Calendrier / matrice agents × ruches	10
4.3.2 Tableau de bord production (fermier / propriétaire)	10
4.3.3 Tableau de bord alertes (responsable)	11
4.3.4 Tableau de bord de synthèse (propriétaire)	11
4.4 Indicateurs de performance (volet automatisé : préparation)	11
4.4.1 Analyse prédictive et surveillance connectée	11
4.4.2 Protocole d'ingestion des mesures	11
4.4.3 Seuils par défaut et logique d'alerte	12
4.5 Grille de synthèse des fonctions	13
4.6 Fonctionnalités complémentaires inspirées des solutions du marché	14
4.7 Fonctions de gestion avancée (inspiration Apiary Book et BeePlus)	14
4.8 Limites connues des solutions existantes et exigences pour les éviter	15

5	Dictionnaire de données et règles de calcul	17
5.1	Conventions de types	17
5.2	Dictionnaire des données (inputs)	17
5.3	Résultats et règles de calcul (outputs)	19
5.4	Requêtes types	19
6	Cas d'utilisation détaillés	20
6.1	UC1 : Planifier et approuver une visite	20
6.2	UC2 : Réaliser une visite et remplir le rapport	20
6.3	UC3 : Consulter le tableau de bord de production	20
6.4	UC4 : Traiter une alerte sanitaire	22
6.5	UC5 : Interroger la quantité de miel via l'API	22
7	Conception (modèles UML)	23
7.1	Diagramme de classes	23
7.2	Vue en couches (package)	23
7.3	Vue de déploiement	25
7.4	Diagramme de séquence (UC2)	25
7.5	Séquence d'authentification (OAuth Google)	25
7.6	Diagramme d'objets	25
7.7	Diagramme d'activité	25
7.8	Diagramme de machine à états	25
7.9	Diagramme de collaboration	25
7.10	Diagramme de composants	30
8	Contraintes et exigences techniques	31
8.1	Architecture	31
8.2	Exigences techniques imposées	31
8.3	Exigences non fonctionnelles	31
8.4	Questions de conception à couvrir	32
8.5	Justification des choix technologiques	32
8.6	Packaging et déploiement	33
9	Enveloppe budgétaire et ressources	34
9.1	Cadre	34
9.2	Ressources mobilisées	34
10	Délais et livrables	35
10.1	Livrables attendus	35
10.2	Grille d'évaluation (barème)	35
10.3	Diagrammes UML à produire	36
10.4	Planning prévisionnel	36
11	Plan de tests et validation	37
11.1	Cas de tests	37
11.2	Matrice de traçabilité	38

12 Référentiel métier et bonnes pratiques apicoles	39
12.1 Facteurs de production du miel	39
12.2 Emplacement et configuration d'un site	39
12.3 Types de ruches et rendement de référence	40
12.4 Classification des modèles de ruches	40
12.4.1 Composition commune d'une ruche à cadres	41
12.5 Protocole d'inspection des colonies	41
12.6 Essaimage et division des colonies	41
12.7 Indicateurs de bien-être et causes de baisse	42
12.8 Incidence sur les seuils paramétrables	42
A Annexe : Structure physique d'une ruche	44
B Glossaire	45
C Références et sources	47
D Annexe : Pile technologique et comparatif des alternatives	49
D.1 Pile technologique retenue	49
D.2 Comparatif justifié des alternatives	50
D.2.1 Serveur d'applications	50
D.2.2 Système de gestion de base de données	50
D.2.3 Style du service web tiers	51
D.2.4 Cartographie et rayons de butinage	51
D.2.5 Bibliothèque de graphiques	51
D.2.6 Accès hors-ligne et mobile	51
E Annexe - Choix technologique : academique et vision marche	52
E.1 Positionnement	52
E.2 Tableau de correspondance : coeur technique	52
E.3 Tableau de correspondance : infrastructure et deploiement	53
E.4 Correspondance des couches	54
E.5 Argumentaire : conformite academique et vision marche	54
F Annexe - Principes SOLID et design patterns	56
F.1 Les cinq principes SOLID	56
F.2 Design patterns retenus	57
F.3 Vue de conception refactorée	58
F.4 Illustration dynamique : patrons Observer et Command	58
F.4.1 Observer (+ Strategy) : declenchement d'une alerte	59
F.4.2 Command : mode hors-ligne et synchronisation differee	59
F.4.3 State : cycle de vie de la ruche	59
F.4.4 Composite : calcul de la quantite de miel	62
F.4.5 Strategy : conversion d'unites heterogenes	62
F.4.6 Adapter : ingestion de capteurs heterogenes	62
F.5 Repartition des patrons par couche	64

G	Annexe - Complements : donnees, IHM, securite et tests	65
G.1	Modele logique de donnees (MLD)	65
G.1.1	Table de correspondance du vocabulaire	65
G.2	Maquettes de l'interface (wireframes)	66
G.3	Matrice des droits d'accès (RBAC)	66
G.4	Registre des risques	66
G.5	Protection des donnees personnelles	67
G.6	Strategie de tests	68
H	Annexe - Intelligence artificielle et analyse predictive	72
H.1	Principes directeurs	72
H.2	Cartographie des usages d'IA	72
H.3	Cas retenu : detection d'anomalie adaptative	73
H.3.1	Formalisation (sans code)	73
H.3.2	Parametres (tous parametrables via ConfigZümm.ini)	74
H.4	Architecture d'integration	75
H.4.1	Le detecteur comme Strategy interchangeable	75
H.4.2	Les traitements lourds comme microservice decouple	75
H.5	Usages anticipes (interfaces specifiees, hors perimetre de developpement)	76
H.6	Ethique, limites et conformite	76
I	Annexe - Securite et robustesse	78
I.1	Modele de menace synthetique	78
I.2	Defense en profondeur	79
I.2.1	Peripherie et transport	79
I.2.2	Application	79
I.2.3	Donnees et exploitation	79
I.3	Grille de durcissement	80
I.4	Robustesse : tests de charge et de fiabilite (k6)	80
I.4.1	Types de tests	80
I.4.2	Scenarios propres a Zümm	81
I.4.3	Seuils et verdict	81
I.4.4	Complement au plan de tests	81
I.5	Checklist de pre-deploiement (go-live)	82
I.6	Conformite et coherence avec les exigences	82

Historique des versions

Version	Date	Auteur	Modifications
0.1	13 juillet 2026	Équipe projet	Structure initiale et transcription du sujet.
0.5	13 juillet 2026	Équipe projet	Référentiel métier apicole et illustrations.
1.0	13 juillet 2026	Équipe projet	Fonctionnalités du marché, exigences anti-défaut, dictionnaire de données, cas d'utilisation, conception UML et plan de tests.

CHAPITRE 1

Contexte et définition du problème

1.1 Présentation générale

Le projet **Zümm** vise à concevoir un système d'information dédié à la gestion et au suivi des activités apicoles. L'apiculture moderne repose sur la surveillance régulière de nombreuses ruches, souvent réparties sur plusieurs sites géographiques et exploitées par différents apiculteurs. Le suivi de la santé des colonies, de leur productivité et des interventions de maintenance constitue une charge organisationnelle importante, aujourd'hui gérée de façon largement manuelle et dispersée.

1.2 Problématique

En l'absence d'un outil centralisé, les exploitants apicoles rencontrent plusieurs difficultés :

- absence de traçabilité structurée des visites et des inspections des ruches
- difficulté à planifier et à coordonner le travail des agents apiculteurs
- manque de visibilité sur la production (miel, poids) et sur la rentabilité de chaque site ou ruche
- détection tardive des anomalies (baisse d'effectif, chute de production, ouverture non planifiée d'une ruche)
- impossibilité d'exploiter automatiquement les mesures issues de capteurs hétérogènes.

1.3 Finalité

Le système doit apporter une réponse structurée en deux volets complémentaires :

1. une **gestion manuelle** des opérations de manutention, de leur planification et de leur suivi (première partie, objet du présent projet)
2. une **supervision distante et automatisée** des indicateurs de performance, alimentée par des capteurs, destinée à être développée dans le cadre d'un projet ultérieur mais dont le système doit anticiper l'intégration.

CHAPITRE 2

Objectifs du projet

2.1 Objectif principal

Fournir une application **multi-niveaux, évolutive et faiblement couplée** permettant de gérer l'ensemble du cycle de vie des ruches, de leur composition physique jusqu'au suivi de production, et d'offrir des tableaux de bord d'aide à la décision aux différents profils d'utilisateurs.

2.2 Objectifs spécifiques

- Assurer les opérations **CRUD** (création, lecture, mise à jour, suppression) sur l'ensemble des entités métier.
- Gérer la hiérarchie *fermier / ferme / site / ruche / corps ou hausse / cadre*.
- Planifier, approuver et suivre les visites d'inspection des agents apiculteurs.
- Produire des **rapports de visite** structurés et exploitables.
- Offrir des **tableaux de bord** filtrables (calendrier agents × ruches, alertes de production, alertes sanitaires, synthèse financière).
- Anticiper l'intégration de mesures **horodatées et géolocalisées** issues de capteurs à unités hétérogènes.
- Garantir **internationalisation, sécurité** et **interopérabilité** (services web).

2.3 Résultats attendus (indicateurs de réussite)

- Support d'au moins **trois langues** (FR, EN, AR) en version de démonstration.
- Détection automatique des dépassements de seuils **paramétrables** (production, effectif).
- Restitution des données de production par ferme, site et ruche sous forme de graphiques.
- Exposition d'une API interrogeable par des applications tierces (Excel, etc.).

CHAPITRE 3

Périmètre du projet

3.1 Domaine couvert

Le périmètre du présent cahier des charges couvre **la première partie** du système : la gestion manuelle des opérations, leur planification et leur suivi, ainsi que la préparation de l'architecture pour l'intégration future des indicateurs automatisés.

3.2 Modèle métier et règles de gestion

Entité	Règles de gestion
Fermier	Peut posséder plusieurs fermes.
Ferme	Peut comporter plusieurs sites d'apiculture.
Site	Contient une ou plusieurs ruches, dispose d'une localisation géographique (latitude, longitude, altitude, date de mise en œuvre, éventuelles dates de déménagement / clôture). Un site peut accueillir des ruches de différents fermiers et de différentes fermes.
Ruche	Composée d'un compartiment de base (socle / corps) et d'au plus 5 étages d'extension (hausses).
Corps / Hausse	Peut accueillir des cadres de cire installés sur des <i>slots</i> identifiés. Le socle/corps doit contenir au moins 1 cadre, la capacité maximale d'un corps comme d'une hausse est de 10 cadres .
Cadre	Occupe un emplacement (slot) unique dans le corps ou la hausse.
Agent apiculteur	Réalise les visites et inspections. Une ruche peut être suivie successivement par plusieurs agents, mais elle est sous la responsabilité d' un seul agent à un instant donné.
Agent superviseur	Approuve le planning de visite des ruches dont il a la charge.

Contrainte de composition : 1 ruche = 1 socle/corps obligatoire + 0 à 5 hausses, chaque corps/hausse = 1 à 10 cadres, un cadre par slot.

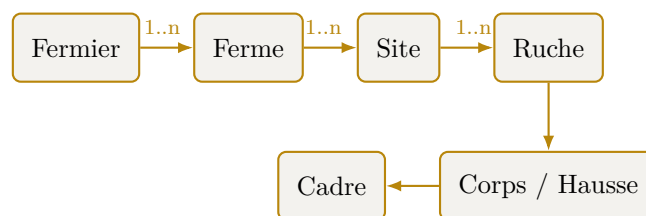


Figure 3.1. Hiérarchie des entités métier du système Zümm.

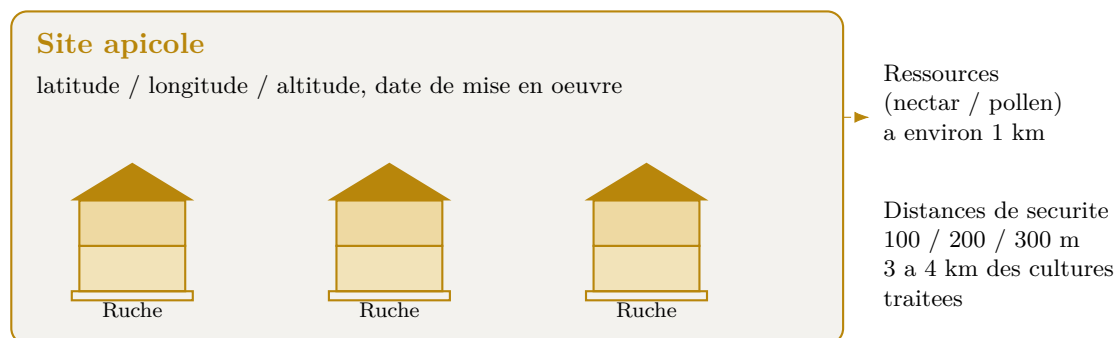


Figure 3.2. Schéma d'un site apicole : plusieurs ruches géolocalisées et contraintes d'emplacement.

3.3 Utilisateurs et droits d'accès

Profil	Rôle et droits
Fermier / Propriétaire	Consulte les tableaux de bord de production, de rentabilité et de synthèse, décide de la clôture d'un site ou du déplacement de ruches.
Agent apiculteur	Réalise les visites, remplit les rapports, exécute les opérations sur les ruches sous sa responsabilité.
Agent superviseur	Approuve les plannings de visite des ruches dont il est responsable.
Responsable / Manager	Surveille les alertes (chute de production ou d'effectif) et déclenche les inspections imminentes.
Administrateur	Gère les paramètres système, les unités de mesure, les seuils et les référentiels.

3.4 Hors périmètre

- L'acquisition physique et le déploiement matériel des capteurs (projet ultérieur).
- L'automatisation temps réel de la collecte des indicateurs (interface prévue mais non implémentée dans cette phase).

CHAPITRE 4

Description fonctionnelle des besoins

4.1 Gestion des entités (CRUD)

Le système doit assurer la création, la consultation, la modification et la suppression de toutes les entités métier : fermiers, fermes, sites, ruches, corps/hausses, cadres, agents, visites et rapports, dans le respect des règles de gestion énoncées au chapitre 3.

4.2 Planification et suivi des visites

- Les visites sont **régulières ou irrégulières**, pour des motifs variés : inspection sanitaire, peuplement par un nouvel essaim, changement de reine, ravitaillement de nourriture, prescription de médicament, subdivision d'essaim, ajout de cadre et/ou d'étage d'extension, collecte de cadres de miel, évaluation de la production, évaluation de l'effectif et du volume de l'essaim.
- Chaque agent suit un **planning de visite approuvé** par l'agent superviseur de la ruche concernée.

4.2.1 Rapport de visite

Chaque visite donne lieu à un rapport contenant : **date, heure, durée, raison, constatations, actions prévues, actions effectuées, recommandations**, une évaluation qualitative de l'effectif / du volume de l'essaim, son état de santé et son niveau de productivité sur une **échelle de 1 à 3**.

4.3 Tableaux de bord

4.3.1 Calendrier / matrice agents × ruches

Matrice croisant **agents et ruches**, déclinée par mois, semaine, saison et année, avec **filtres** et regroupements par site, par agent et par agent responsable. Un clic sur une case représentant une visite planifiée affiche un **pop-up** résumant ce qui est prévu (date, heure, raison, actions) et, si la visite est déjà exécutée, ce qui a été réalisé ainsi que la date et l'heure effectives.

4.3.2 Tableau de bord production (fermier / propriétaire)

Met en évidence les couples **site × ruche** dont la production (poids) a dépassé un **seuil paramétrable**, signalant soit une collecte de miel imminente, soit une intervention d'extension et/ou de division d'essaim pour favoriser l'expansion de la colonie.

4.3.3 Tableau de bord alertes (responsable)

Signale les couples **site** × **ruche** dont la production ou l'effectif baissent de façon inhabituelle au-delà d'un seuil paramétrable, déclenchant une **alerte** et une inspection imminente.

4.3.4 Tableau de bord de synthèse (propriétaire)

Graphiques, courbes et histogrammes représentant :

- la quantité produite par ferme et par site
- le nombre d'interventions par ferme, par site et par ruche
- le **retour sur investissement** (rapport coûts / production) pour arbitrer le sort de chaque site ou ruche (clôture, déplacement).

4.4 Indicateurs de performance (volet automatisé : préparation)

Les mesures sont **horodatées et géolocalisées**, exprimées dans des unités **paramétrables** (internationalisation des systèmes d'unités). Les mesures d'un même indicateur ne partagent pas nécessairement la même unité, les capteurs étant hétérogènes. Indicateurs concernés :

- poids du cadre, du socle et de l'étage d'extension
- température et humidité, à l'intérieur et à l'extérieur de la ruche
- effectif des mouvements d'entrée et de sortie des abeilles
- niveau de bruit / bourdonnement, intérieur et extérieur
- niveau de luminosité, intérieur et extérieur
- vitesse et direction du vent à l'extérieur
- état d'ouverture de la ruche (intervention, orage, vol, attaque animale, etc.).

4.4.1 Analyse prédictive et surveillance connectée

Les solutions de surveillance connectée (precision beekeeping) telles qu'Arnia, BroodMinder ou BeeHero montrent que l'exploitation avancée des mêmes indicateurs permet d'anticiper les événements de la colonie. Le modèle de données doit rester ouvert à ces traitements, prévus pour le volet automatisé.

- détection acoustique de l'essaimage 1 à 5 jours à l'avance par analyse de la signature sonore de la colonie
- confirmation de la présence de la reine à partir du bourdonnement
- détection d'ouverture, de choc ou de basculement de la ruche par accéléromètre (vol, chute, attaque animale), en complément de l'indicateur d'ouverture
- orientation de la ruche par boussole électronique
- pesée continue par balance pour suivre le flux de nectar et la dynamique de production
- analyse dans le cloud par intelligence artificielle des signatures acoustiques pour corréler son et activité de la ruche.

4.4.2 Protocole d'ingestion des mesures

Le volet automatisé reste hors périmètre de développement, mais l'interface d'ingestion est spécifiée dès maintenant pour que le modèle de données et l'architecture l'anticipent.

- **Transport** : une passerelle terrain transmet les mesures au serveur, au choix selon la connectivité, par **REST sur HTTPS** (POST /api/mesures) ou par **MQTT** (publication sur le topic `zumm/{rucheId}/{indicateur}`).
- **Format** : **JSON**, une mesure portant l'identifiant de la ruche, le type d'indicateur, la valeur, l'unité, l'horodatage et la géolocalisation. L'envoi par lot est un tableau de mesures.
- **Fréquence** : paramétrable par indicateur (par exemple poids toutes les 15 min, température toutes les 5 min, acoustique échantillonnée en continu), afin de maîtriser la volumétrie.
- **Robustesse** : ingestion **asynchrone** par file d'attente, avec **idempotence** sur la clé (ruche, indicateur, horodatage) pour rejouer sans doublon. En zone reculée, mise en tampon locale sur la passerelle puis synchronisation différée.
- **Sécurité** : passerelle authentifiée (clé ou certificat), canal TLS. Rejet des valeurs hors plage physique plausible (garde-fou).

POST /api/mesures Content-Type: application/json

```
{
  "rucheId": 42, "typeIndicateur": "TEMPERATURE_INTERNE",
  "valeur": 34.8, "unite": "degC",
  "horodatage": "2026-07-09T14:05:00Z",
  "latitude": 36.806, "longitude": 10.181
}
```

4.4.3 Seuils par défaut et logique d'alerte

Les seuils sont stockés dans la table **SEUIL** (paramétrables, chapitre 5). Les valeurs ci-dessous sont des **valeurs par défaut de référence**, ancrées dans le protocole apicole (chapitre 12).

Indicateur	Seuil par défaut	Alerte déclenchée
Température interne (couvain)	< 32 °C ou > 36 °C	Risque thermique sur le couvain
Humidité interne	> 80 %	Excès d'humidité, risque sanitaire
Poids : gain de la hausse	> seuil de production	Collecte de miel imminente
Poids : chute soudaine	> 1,5 kg en moins d'1 h	Essaimage, vol, chute ou attaque
Production ou effectif	Baisse relative > 20 % vs période précédente	Inspection sanitaire imminente
Activité entrée/sortie	≈ 0 en journée favorable	Colonie affaiblie ou reine absente
État d'ouverture	Ouvert hors visite planifiée	Intrusion ou perturbation
Signature acoustique	Motif d'essaimage détecté	Pré-alerte d'essaimage (1 à 5 j)

Règle d'évaluation : à chaque mesure, la valeur est convertie vers l'unité de référence (formule de conversion, §5.3), puis comparée au seuil paramétrable. Pour limiter les *fausses alertes*, une alerte n'est levée que si le dépassement **persiste sur N mesures consécutives** (anti-rebond / hystérésis). Toute alerte reste une **aide à la décision** confirmée par le rapport de visite : l'agent conserve la validation finale.

4.5 Grille de synthèse des fonctions

Fonction	Description	Priorité
CRUD entités	Gestion complète du modèle métier	Haute
Planning de visites	Planification + approbation par le superviseur	Haute
Rapports de visite	Saisie structurée des constats et actions	Haute
Calendrier agents×ruches	Matrice filtrable avec pop-up de résumé	Haute
Alertes production	Détection des dépassements de seuil	Haute
Alertes sanitaires	Détection des baisses anormales d'effectif/production	Haute
Synthèse & ROI	Graphiques coûts/production, aide à la décision	Moyenne
Indicateurs capteurs	Modèle de données prêt pour l'automatisation	Moyenne
<i>Exigences anti-défaut (tirées des limites des solutions existantes)</i>		
Déploiement autonome	Installation J2EE sans abonnement ni service tiers obligatoire	Haute
Mode hors-ligne	Saisie sans connexion et synchronisation différée	Haute
Authentification simple	OAuth Google et parcours de saisie terrain rapide	Haute
Modularité configurable	Activation des modules par profil via ConfigZümm.ini	Moyenne
Portabilité des données	Export CSV et TXT, service web API et sauvegarde, sans verrouillage	Haute
Aide contextuelle	Interface simple et cohérente, guidage de l'utilisateur	Moyenne
Parité web et mobile	Mêmes fonctions sur tous les supports	Moyenne
Internationalisation	Fichier XML FR, EN et AR, extensible à d'autres langues	Haute
Ouverture matérielle	Capteurs hétérogènes et unités paramétrables	Moyenne
Ingestion asynchrone	Mesures horodatées différées, volet manuel autonome	Moyenne
Validation humaine	Seuils paramétrables et confirmation par le rapport de visite	Haute

Fonction	Description	Priorité
Sécurité des échanges	SSL et certificats X.509, accès authentifiés	Haute
Aide à la décision	Alertes indicatives, l'agent conserve la validation finale	Moyenne

4.6 Fonctionnalités complémentaires inspirées des solutions du marché

Afin de renforcer l'ergonomie et la valeur d'usage, le système s'inspire des bonnes pratiques des applications apicoles de référence telles que HiveTracks. Ces fonctionnalités étendent les besoins déjà décrits sans les remplacer.

Fonction	Description	Priorité
Photos d'inspection	Attacher une ou plusieurs photos à chaque rapport de visite pour un suivi visuel de la colonie et des cadres.	Haute
Saisie vocale assistée	Dictier le compte rendu de visite, la transcription alimentant automatiquement les champs du rapport.	Basse
Contexte météo	Afficher la météo locale de la semaine à partir de la localisation du site pour éclairer les décisions d'intervention.	Moyenne
Carte et rayons de butinage	Positionner les ruches sur une carte et tracer des rayons de butinage (par exemple 1, 2 et 3 km, unités paramétrables) pour visualiser l'environnement accessible aux abeilles.	Moyenne
Liste de tâches et rappels	Gérer une liste de tâches et un calendrier de rappels (nourrissement, inspection, statut de la reine) pour ne manquer aucune échéance.	Haute
Suivi de la reine	Enregistrer l'historique du statut de la reine (présence, âge, remplacement, marquage) au fil des visites.	Haute
Récolte et QR code	Saisir les récoltes de miel et générer un QR code par lot présentant les notes de dégustation, la provenance et les photos, pour la traçabilité et la valorisation.	Moyenne
Accès multi-supports	Offrir un accès web et une expérience adaptée aux terminaux mobiles pour la saisie terrain.	Moyenne

Cohérence avec les exigences : les rayons de butinage et la météo réutilisent la géolocalisation des sites et le mécanisme d'unités paramétrables. Les rappels s'appuient sur le planning de visite existant, et le suivi de la reine enrichit le rapport de visite déjà défini.

4.7 Fonctions de gestion avancée (inspiration Apiary Book et Bee-Plus)

Les applications de gestion apicole Apiary Book et BeePlus mettent en avant des fonctions d'organisation et de traçabilité qui complètent le périmètre du système.

Fonction	Description	Priorité
Mode hors-ligne et synchronisation	Saisir les données terrain sans connexion et les synchroniser automatiquement au retour du réseau.	Haute
Suivi des traitements	Conserver l'historique des médicaments et traitements sanitaires appliqués à chaque ruche.	Haute
Gestion du matériel et inventaire	Suivre le matériel (corps, hausses, cadres, ruches) et son affectation aux sites.	Moyenne
Suivi financier	Enregistrer coûts et revenus par ferme, site et ruche, en appui du calcul du retour sur investissement.	Moyenne
Export des données	Exporter les enregistrements aux formats CSV et TXT pour une analyse externe.	Moyenne
Détection de maladies	Aider à l'identification des maladies à partir des constatations de visite.	Basse
Sauvegarde des données	Sauvegarder et restaurer les données dans le cloud.	Moyenne

Cohérence avec les exigences : le suivi financier alimente le tableau de bord de retour sur investissement, l'export CSV prolonge le service web API, et le suivi des traitements enrichit le motif de visite prescription de médicament.

4.8 Limites connues des solutions existantes et exigences pour les éviter

L'analyse des retours d'expérience sur les solutions du marché (HiveTracks, Apiary Book, BeePlus) et sur les systèmes de surveillance connectée (Arnia, BroodMinder, BeeHero) fait ressortir des limites récurrentes. Le tableau suivant met chaque limite en regard de l'exigence retenue pour l'éviter dans Zümm.

Limite observée	Exigence retenue pour l'éviter
Abonnement payant et paywall	Système autodéployable en J2EE, sans abonnement obligatoire, l'ensemble des fonctions de gestion reste accessible.
Fonctionnement hors-ligne limité	Mode hors-ligne robuste avec synchronisation différée, la saisie terrain n'exige pas de connexion permanente.
Création de compte imposée et friction	Authentification simple par OAuth Google et parcours de saisie rapide, l'ergonomie prime.
Surcharge de fonctions inutiles	Modules activables par profil et par le fichier de configuration ConfigZümm.ini, grâce à l'architecture faiblement couplée.
Dépendance et perte des données	Données maîtrisées par l'utilisateur, export CSV et TXT, service web API et sauvegarde, sans verrouillage propriétaire.
Interface dense et courbe d'apprentissage	Interface simple, cohérente et conviviale avec aide contextuelle, conformément aux exigences d'ergonomie.
Fonctions inégales selon la plateforme	Parité fonctionnelle entre accès web et mobile grâce à l'architecture multi-niveaux.

Limite observée	Exigence retenue pour l'éviter
Solutions uniquement anglophones	Internationalisation par fichier XML, au moins FR, EN et AR, ouverte à d'autres langues.
Coût élevé du matériel de capteurs	Ouverture aux capteurs hétérogènes et bon marché avec unités paramétrables, sans verrouillage matériel.
Autonomie et connectivité en zone reculée	Ingestion asynchrone de mesures horodatées, le volet manuel fonctionne sans capteurs.
Fausses alertes et fiabilité des capteurs	Seuils paramétrables et validation humaine par le rapport de visite, les alertes restent une aide à la décision.
Confidentialité et sécurité des données	Chiffrement SSL et certificats X.509, authentification des accès.
Surconfiance dans l'automatisation	Positionnement en aide à la décision, l'agent apiculteur conserve la validation finale.

CHAPITRE 5

Dictionnaire de données et règles de calcul

Ce chapitre répond à la question des inputs et des outputs du système. Il définit les données manipulées, leurs types et les formules de calcul des résultats.

5.1 Conventions de types

Les types utilisés sont : entier, décimal, texte, date, heure, datetime, booléen, énumération et référence (clé étrangère vers une autre entité). Les grandeurs physiques sont associées à une unité paramétrable pour permettre l'internationalisation des systèmes de mesure.

5.2 Dictionnaire des données (inputs)

Entité	Attribut	Type	Description
Fermier	id, nom, contact	entier, texte	Propriétaire exploitant.
Ferme	id, nom, fermier	entier, texte, référence	Exploitation rattachée à un fermier.
Site	id, nom	entier, texte	Emplacement d'apiculture.
Site	latitude, longitude	décimal (degrés)	Géolocalisation.
Site	altitude	décimal (unité paramétrable)	Altitude du site.
Site	dateMiseEnOeuvre, dateDemenagement, dateCloture	date	Cycle de vie du site, deux dernières optionnelles.
Ruche	id, modele	entier, texte	Ruche hébergée par un site.
Ruche	nbHausses	entier (0 à 5)	Nombre d'étages d'extension.
Ruche	ferme	référence	Ferme propriétaire (distincte du site d'emplacement).

Entité	Attribut	Type	Description
Ruche	etat	énumération	État du cycle de vie (créée, peuplée, active, en division, en collecte, clôturée), voir figure 7.8.
Ruche	agentResponsable	référence	Agent responsable à l'instant donné.
Compartiment	id, type	entier, énumération	Compartiment de la ruche : corps ou hausse.
Compartiment	capaciteCadres	entier (1 à 10)	Nombre de slots de cadres.
Cadre	id, slot, type	entier, énumération	Cadre installé sur un slot identifié.
Cadre	poids	décimal (unité paramétrable)	Poids du cadre, base du calcul de la quantité de miel.
Agent	id, nom, role	entier, texte, énumération	Apiculteur, superviseur, responsable ou administrateur.
Planning	id, ruche, agent	entier, référence	Planning de visite.
Planning	datePrevue, statutApprobation	date, énumération	Date prévue et approbation du superviseur.
Planning	superviseur	référence	Agent superviseur qui approuve le planning.
Visite	id, ruche, agent	entier, référence	Visite réalisée.
Visite	planning	référence	Planning approuvé à l'origine de la visite (optionnel).
Visite	date, heure, duree	date, heure, entier (min)	Horodatage et durée.
Visite	raison	énumération	Motif de la visite.
Visite	constatations, actionsPrevues, actionsEffectuees, recommandations	texte	Contenu du rapport.
Visite	effectifQualitatif, etatSante	énumération	Évaluation de l'essaim.
Visite	productivite	entier (1 à 3)	Niveau de productivité.
Photo	id, url, visite	entier, texte, référence	Photo attachée à un rapport de visite.
Mesure	id, ruche, typeIndicateur	entier, référence, énumération	Mesure d'un indicateur.
Mesure	valeur, unite	décimal, texte	Valeur et unité paramétrable.
Mesure	horodatage, latitude, longitude	datetime, décimal	Mesure horodatée et géolocalisée.
Récolte	id, ruche, date	entier, référence, date	Récolte de miel.

Entité	Attribut	Type	Description
Récolte	quantiteMiel, lotQR	décimal (unité paramétrable), texte	Quantité et identifiant de lot.
Seuil	id, type, valeur, unite	entier, énumération, décimal, texte	Seuil paramétrable du système.

Schéma de référence : le dictionnaire ci-dessus est la vue métier des données. Sa traduction relationnelle normalisée (clés, types SQL et contraintes CHECK) constitue le **modèle logique de données** (MLD), qui fait foi pour l'implémentation et est présenté à l'annexe G (figure G.1). La correspondance des noms entre le dictionnaire, le diagramme de classes et le MLD est donnée par la table G.1.

5.3 Résultats et règles de calcul (outputs)

- **Quantité de miel d'une ruche** : somme des poids des cadres de miel des hausses, diminuée de la tare des éléments.

$$\text{QuantiteMiel}(r) = \sum_{c \in \text{cadresMiel}(r)} \text{poids}(c) - \text{tare}(r)$$

Cette valeur est exposée par le service web via `getZummHoneyActualQuantity(zummID)`.

- **Retour sur investissement** d'un site ou d'une ruche :

$$\text{ROI} = \frac{\text{Production valorisée}}{\text{Coûts des interventions}}$$

- **Alerte de collecte** : déclenchée si la production dépasse le seuil paramétrable.

$$\text{Production}(r) > \text{SeuilProduction} \Rightarrow \text{alerte collecte ou extension}$$

- **Alerte de baisse** : déclenchée si la baisse relative dépasse le seuil.

$$\frac{V_{\text{préc}} - V_{\text{actuel}}}{V_{\text{préc}}} > \text{SeuilBaisse} \Rightarrow \text{alerte inspection}$$

- **Conversion d'unités** vers l'unité de référence, pour comparer des mesures hétérogènes :

$$V_{\text{référence}} = V_{\text{mesure}} \times \text{facteurConversion}(unite)$$

5.4 Requêtes types

Les résultats s'appuient sur des requêtes exécutées via JDBC, par exemple pour la quantité de miel d'une ruche : `SELECT SUM(quantiteMiel) FROM recolte WHERE ruche = zummID`.

CHAPITRE 6

Cas d'utilisation détaillés

Cette partie décrit comment le système réalise les opérations des utilisateurs. Les acteurs sont le fermier, l'agent apiculteur, l'agent superviseur, le responsable, l'administrateur et les applications tierces.

Le diagramme de cas d'utilisation global (figure 6.1) présente l'ensemble des acteurs, la généralisation de l'agent superviseur à partir de l'agent apiculteur, ainsi que toutes les dépendances entre cas (relations d'inclusion et d'extension).

6.1 UC1 : Planifier et approuver une visite

- **Acteur** : agent superviseur.
- **Préconditions** : la ruche existe et l'agent est authentifié.
- **Scénario nominal** : l'agent propose une date de visite, le superviseur examine le planning puis approuve, le système enregistre le statut approuvé.
- **Alternatives** : le superviseur refuse et demande une nouvelle date.
- **Postcondition** : la visite est planifiée et visible dans le calendrier.

6.2 UC2 : Réaliser une visite et remplir le rapport

- **Acteur** : agent apiculteur responsable de la ruche.
- **Préconditions** : une visite est planifiée et approuvée.
- **Scénario nominal** : l'agent renseigne date, heure, durée, raison, constatations, actions prévues et effectuées, recommandations, puis évalue l'effectif, l'état de santé et la productivité de 1 à 3.
- **Alternatives** : saisie hors-ligne puis synchronisation différée, ajout de photos.
- **Postcondition** : le rapport est enregistré et la case du calendrier passe au statut exécuté.

6.3 UC3 : Consulter le tableau de bord de production

- **Acteur** : fermier ou propriétaire.
- **Préconditions** : des mesures de production existent.
- **Scénario nominal** : le fermier filtre par ferme, site et période, le système affiche les ruches dont la production dépasse le seuil et propose collecte ou extension.
- **Postcondition** : le fermier décide d'une intervention.

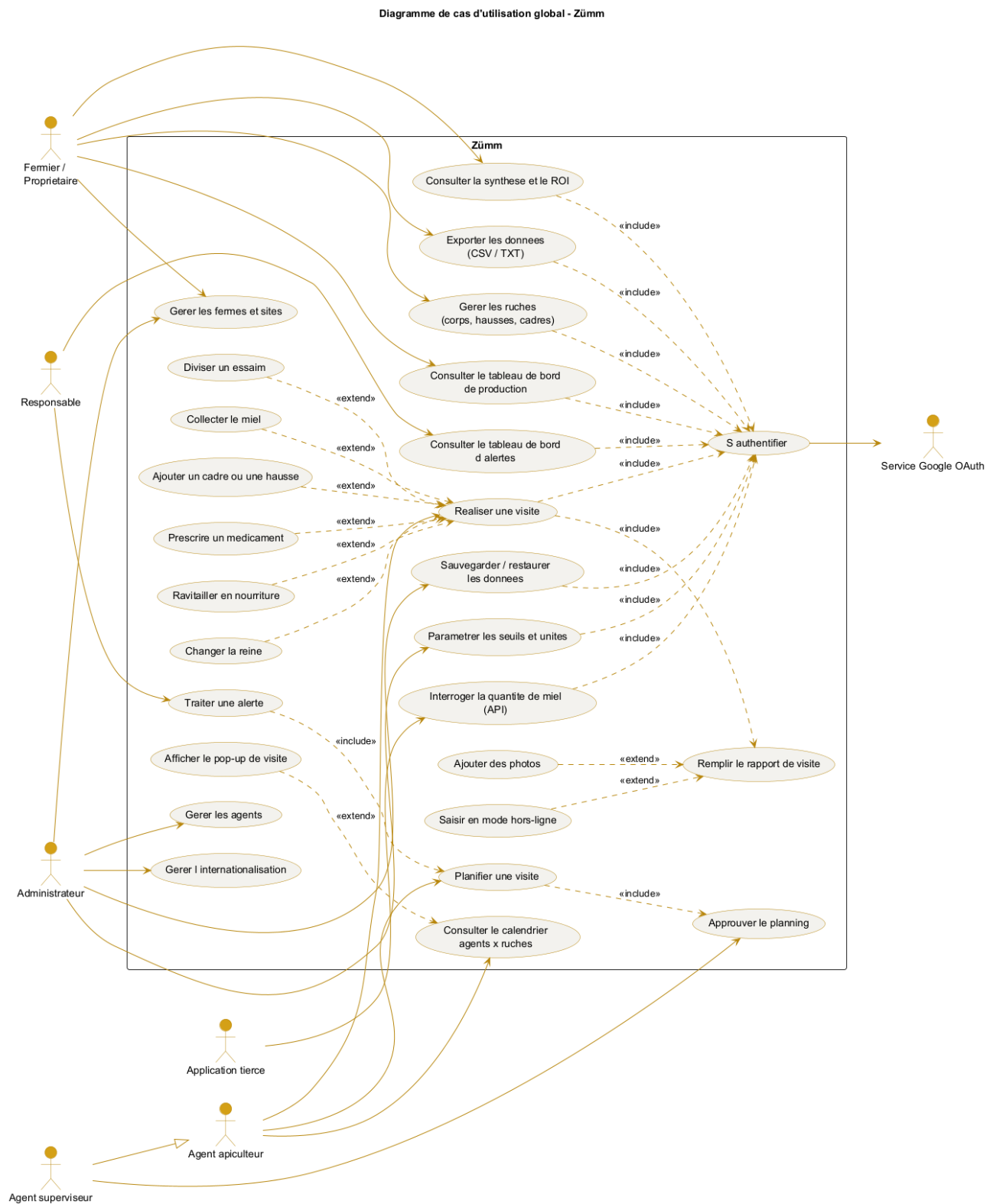


Figure 6.1. Diagramme de cas d'utilisation global avec acteurs et dépendances (include et extend).

6.4 UC4 : Traiter une alerte sanitaire

- **Acteur** : responsable.
- **Préconditions** : une baisse anormale a été détectée.
- **Scénario nominal** : le système signale la ruche concernée, le responsable planifie une inspection imminente.
- **Postcondition** : une visite d'inspection est planifiée.

6.5 UC5 : Interroger la quantité de miel via l'API

- **Acteur** : application tierce (Excel ou autre).
- **Préconditions** : l'application dispose d'un accès sécurisé au service web.
- **Scénario nominal** : l'application appelle `getZummHoneyActualQuantity(zummID)`, le service calcule et retourne la quantité de miel.
- **Postcondition** : la valeur est restituée au format du service web XML.

CHAPITRE 7

Conception (modèles UML)

Le dossier de conception comprend les diagrammes UML suivants. Chacun est décrit par son contenu attendu afin de guider la modélisation.

Diagramme	Contenu attendu
Cas d'utilisation	Acteurs (fermier, agent, superviseur, responsable, administrateur, application tierce) et cas principaux (voir figure 6.1).
Classes	Entités du dictionnaire de données et leurs relations (Fermier 1 vers n Ferme, Ferme 1 vers n Site, Site 1 vers n Ruche, Ruche 1 vers n Corps ou hausse, Corps ou hausse 1 vers n Cadre) et méthodes de service.
Objets	Instantané d'un site contenant trois ruches peuplées avec leurs cadres.
Séquence	Déroulé de la réalisation d'une visite (Agent, IHM JSP, Servlet, EJB, JDBC, base de données).
Activité	Processus de planification, approbation, exécution et clôture d'une visite.
Machine à états	Cycle de vie d'une ruche (créée, peuplée, active, en division, en collecte, clôturée).
Interaction ou collaboration	Même scénario que la séquence sous forme de collaboration entre objets.
Package	Couches présentation, contrôle, métier, accès aux données, internationalisation, configuration et services web.
Composant	Composants applicatifs (application web, service OAuth, service API, module i18n, module configuration, base de données).
Déploiement	Répartition sur le navigateur, le serveur d'applications J2EE, le SGBD, le service Google OAuth et les capteurs.

7.1 Diagramme de classes

Le diagramme de classes, présenté en alignement horizontal, décrit les entités métier, leurs attributs, la méthode de service `getZummHoneyActualQuantity` et les relations avec leurs cardinalités.

7.2 Vue en couches (package)

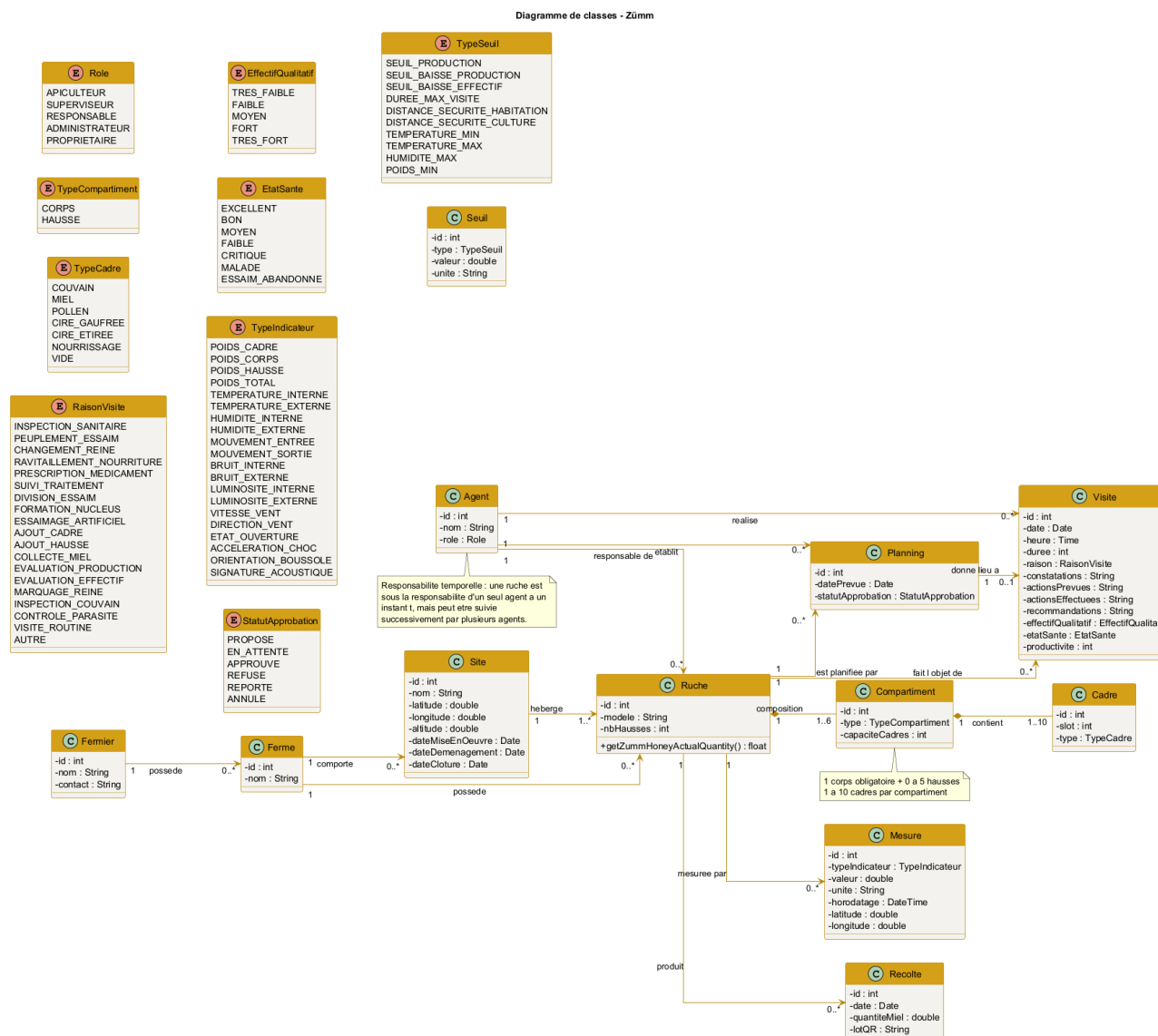
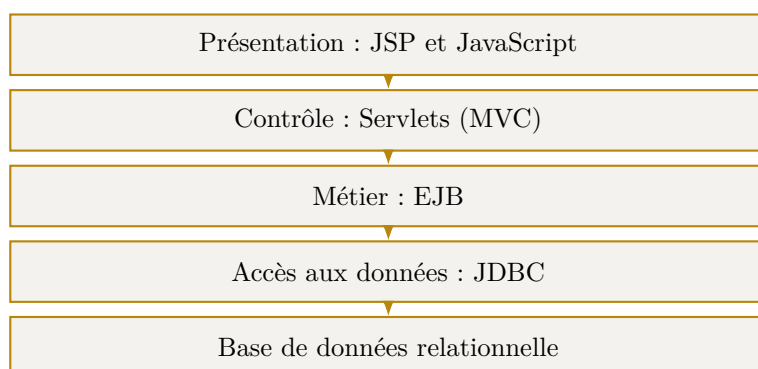


Figure 7.1. Diagramme de classes du système Zümm (alignement horizontal).



i18n XML / ConfigZumm.i

Figure 7.2. Architecture multi-niveaux en couches faiblement couplées.

7.3 Vue de déploiement

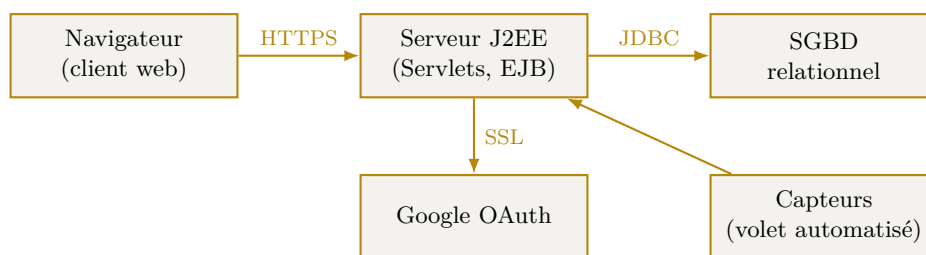


Figure 7.3. Diagramme de déploiement du système.

7.4 Diagramme de séquence (UC2)

Le diagramme de séquence ci-dessous détaille le cas d'utilisation UC2, réaliser une visite et remplir le rapport. Il montre le passage à travers les couches (agent, IHM, servlet, EJB, base de données) ainsi que les cas en ligne et hors-ligne avec synchronisation différée.

7.5 Séquence d'authentification (OAuth Google)

Le parcours d'authentification suit le flot *Authorization Code* d'OAuth 2.0. Le navigateur ne transporte jamais le secret client : l'échange du code d'autorisation contre le jeton d'accès se fait de **serveur à serveur**, sur un canal SSL authentifié par certificat X.509. La session applicative est ensuite ouverte au moyen d'un cookie sécurisé (`HttpOnly`, `Secure`), et le profil renvoyé par Google est associé à un **Agent** existant (ou en crée un en attente d'habilitation). En démonstration, un repli d'authentification locale est prévu en cas d'indisponibilité du fournisseur.

7.6 Diagramme d'objets

Le diagramme d'objets présente un instantané d'un site hébergeant trois ruches, avec le détail de la composition d'une ruche en corps, hausse et cadres.

7.7 Diagramme d'activité

Le diagramme d'activité modélise le processus de planification, d'approbation et de réalisation d'une visite, avec les variantes en ligne et hors-ligne.

7.8 Diagramme de machine à états

Le diagramme de machine à états décrit le cycle de vie d'une ruche, de sa création à sa clôture.

7.9 Diagramme de collaboration

Le diagramme de collaboration reprend le scénario UC2 sous forme de messages numérotés échangés entre les objets.

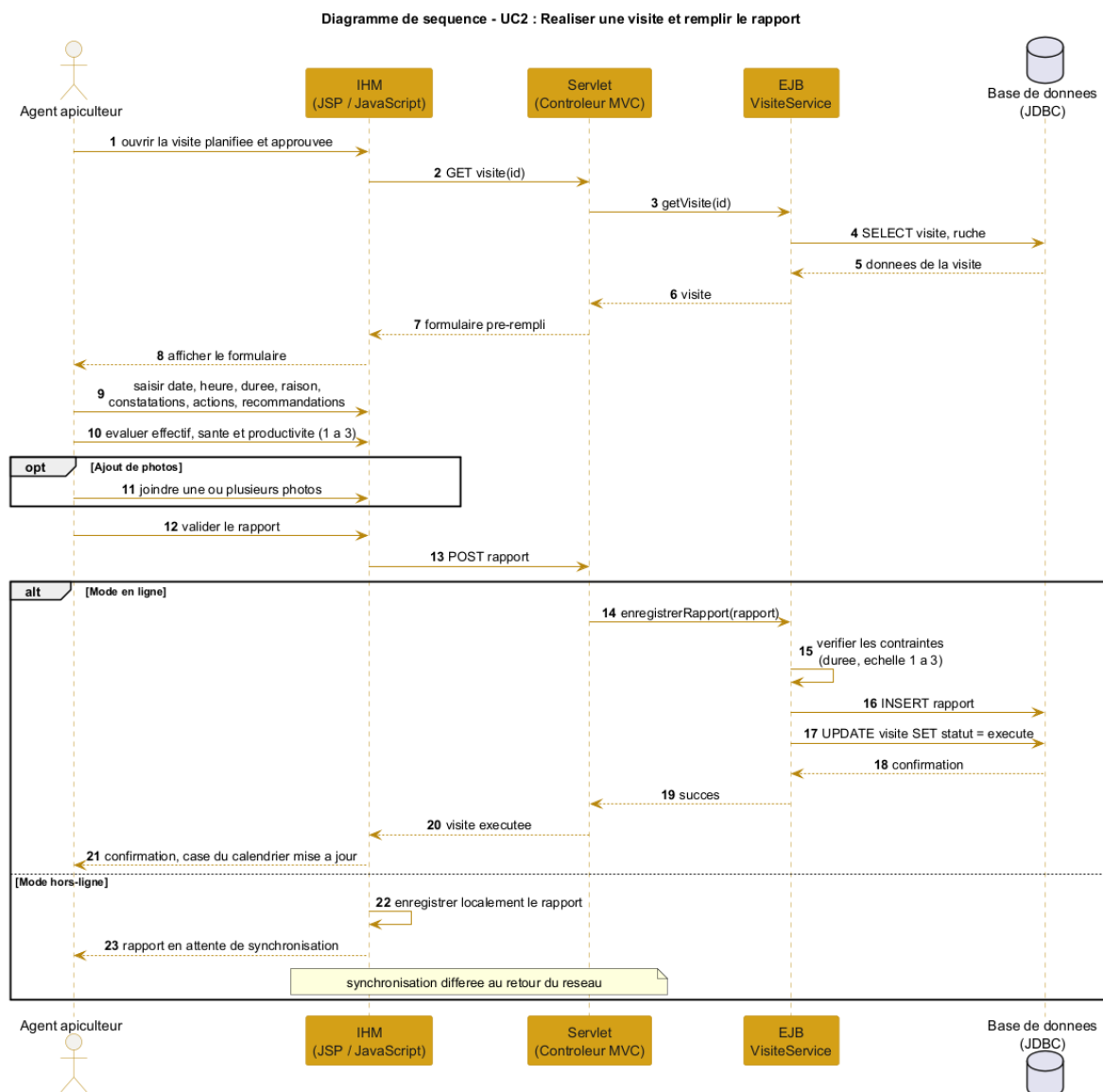


Figure 7.4. Diagramme de séquence du cas d'utilisation UC2 (réaliser une visite).

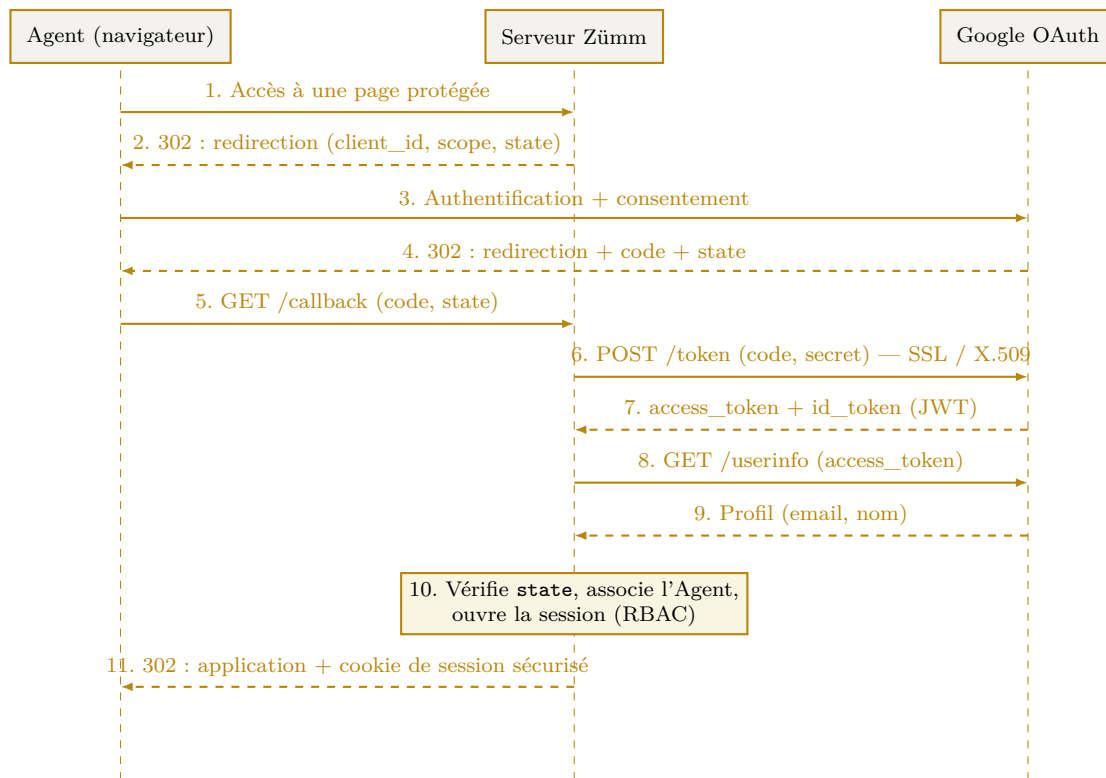


Figure 7.5. Séquence d'authentification OAuth 2.0 (Authorization Code) avec Google.

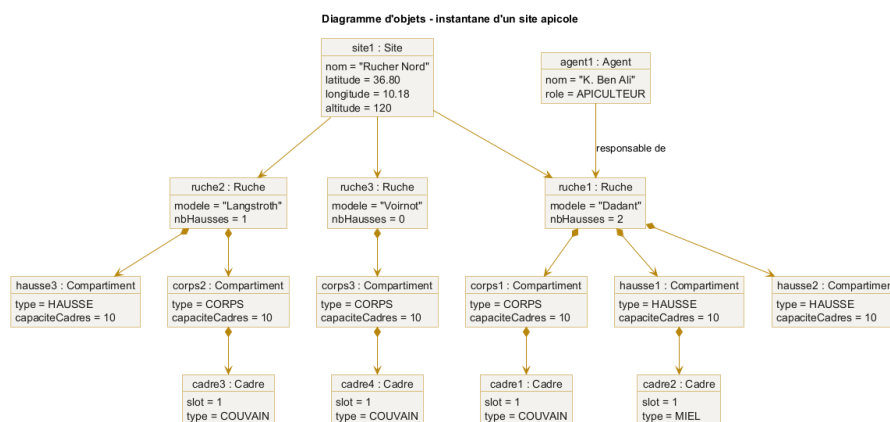


Figure 7.6. Diagramme d'objets, instantané d'un site apicole.

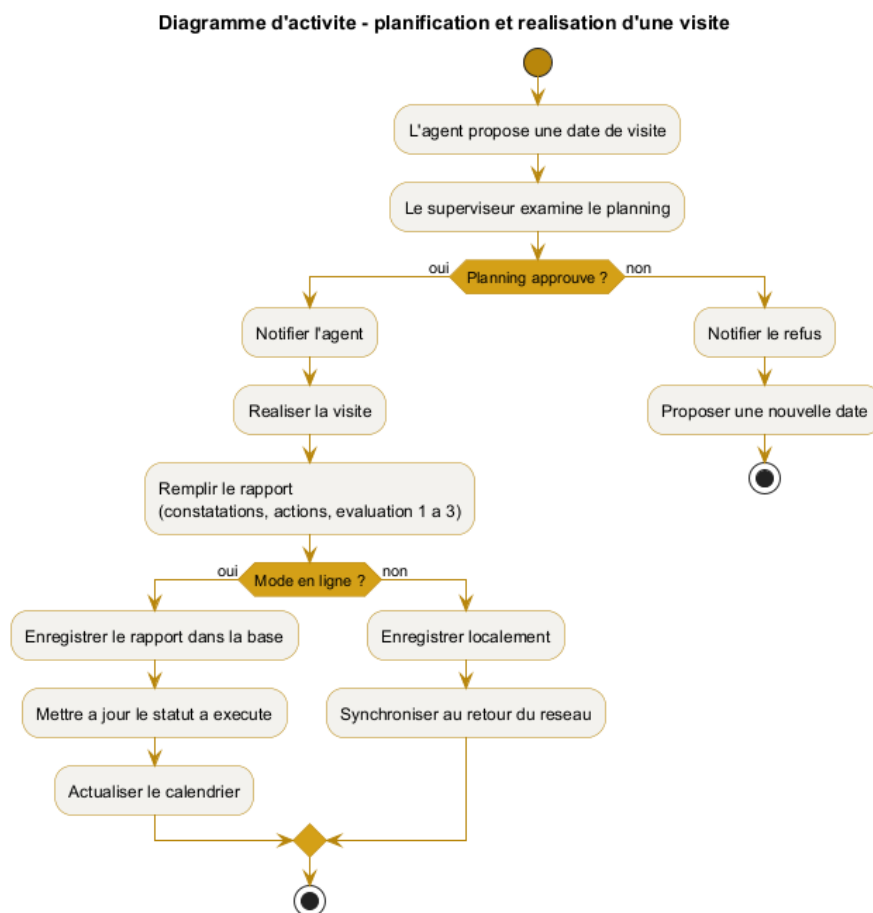


Figure 7.7. Diagramme d'activité du processus de visite.

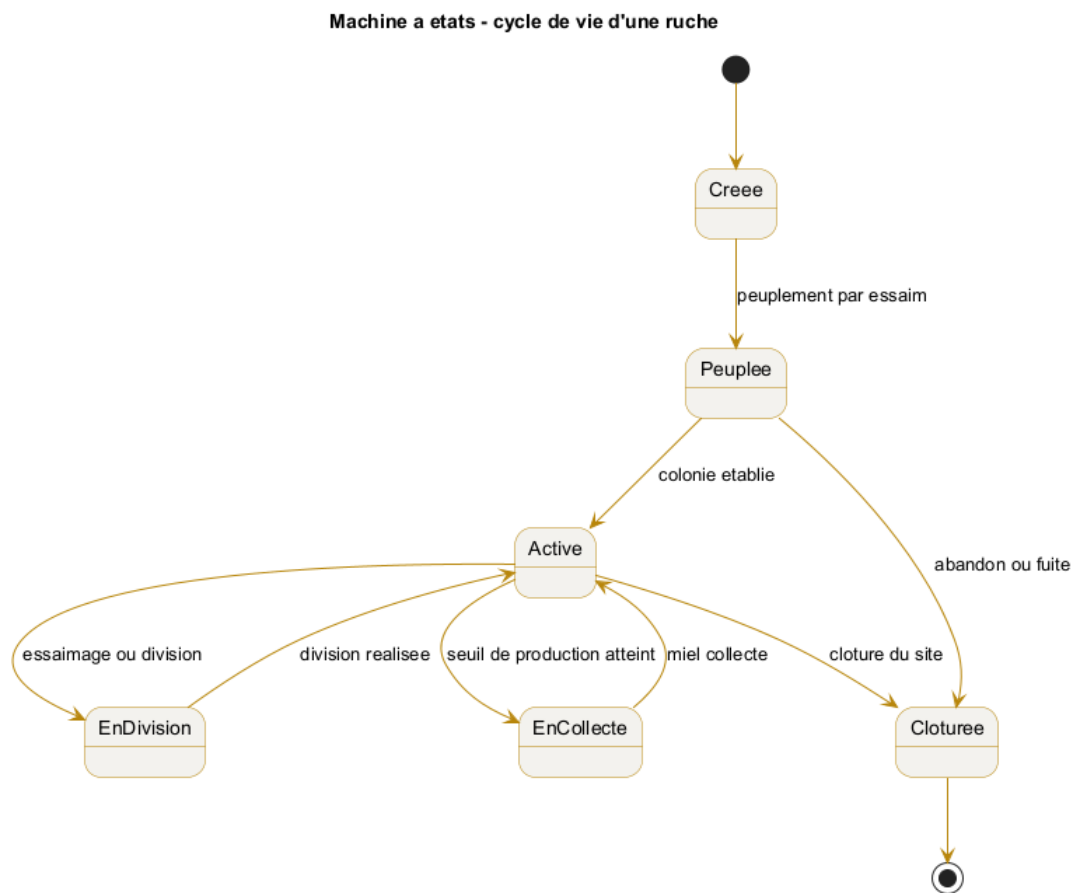


Figure 7.8. Machine à états du cycle de vie d'une ruche.

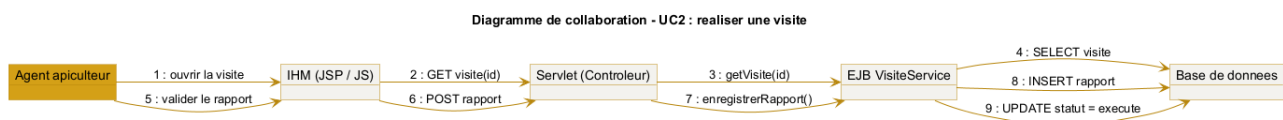


Figure 7.9. Diagramme de collaboration du cas d'utilisation UC2.

7.10 Diagramme de composants

Le diagramme de composants montre l'organisation des composants applicatifs, leurs interfaces et leurs dépendances, dont l'interface de service exposant la quantité de miel.

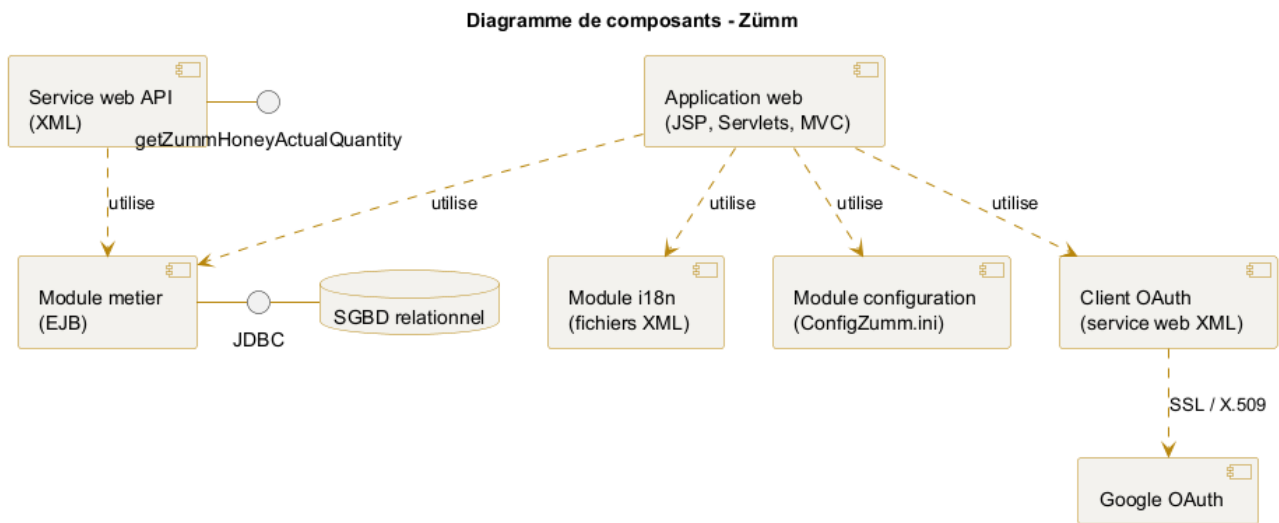


Figure 7.10. Diagramme de composants du système Zümm.

CHAPITRE 8

Contraintes et exigences techniques

8.1 Architecture

Architecture **scalable, évolutive, flexible, multi-niveaux et faiblement couplée**, s'appuyant sur **J2EE** comme infrastructure principale.

8.2 Exigences techniques imposées

Exigence	Description
MVC	Séparation modèle / vue / contrôleur.
Servlet	Traitement des requêtes côté serveur.
JSP	Front-end de présentation.
EJB	Composants métier.
JDBC	Accès aux données.
Internationalisation	Textes de l'IHM externalisés dans un fichier XML, ouverture à la traduction hors-ligne sans limite de langues, FR, EN et AR au minimum en démonstration.
Configuration	Paramétrage par fichier texte <code>ConfigZumm.ini</code> .
Sécurité inter-composants	Certificats X.509 et protocole SSL .
Authentification utilisateur	OAuth Google via un service web XML et son API.
API tierce	Service web XML exposant <code>float getZummHoneyActualQuantity(int zummID)</code> pour récupérer la quantité de miel d'une ruche.
Front-end	JavaScript et frameworks libres, dans le respect de l'ergonomie imposée.

8.3 Exigences non fonctionnelles

- **Ergonomie, convivialité et simplicité** d'utilisation, consistance de l'interface et qualité du design infographique (importance capitale).
- **Évolutivité** : ouverture à l'ajout de langues, d'unités et d'indicateurs.
- **Interopérabilité** : services web interrogeables par des applications tierces.

- **Sécurité** : chiffrement des échanges et authentification forte.
- **Qualité de conception** : respect des principes **SOLID** et recours aux **design patterns** (patrons de conception) pour garantir un couplage faible et l'évolutivité, comme détaillé en annexe F.

Principes de conception retenus : l'architecture applique les cinq principes **SOLID** (responsabilité unique, ouvert/fermé, substitution de Liskov, ségrégation des interfaces, inversion de dépendance) et un ensemble de **design patterns** (Composite, State, Strategy, Observer, Command, Factory Method, Builder, Adapter, Facade, Singleton, Proxy). Leur application au système, justifiée « avant / après », est détaillée dans l'annexe F.

8.4 Questions de conception à couvrir

La conception devra répondre explicitement aux questions suivantes :

1. Quels sont les **inputs** (liste, dictionnaire de données et types) et **outputs** (liste, dictionnaire, types, formules et requêtes de calcul) du système ?
2. De quoi se compose la solution ?
3. Quels sont les utilisateurs types et leurs droits d'accès aux fonctionnalités ?
4. Comment le système réalise-t-il les opérations utilisateur ?
5. Comment le système est-il packagé et déployé ?
6. Comment les éléments interagissent-ils et dans quel ordre / chronologie ?
7. Pourquoi, comment et où telle technologie a-t-elle été utilisée ?

8.5 Justification des choix technologiques

Le tableau suivant répond au pourquoi, au comment et au où de chaque technologie imposée. Les **implémentations concrètes** de ce socle (serveur d'applications, SGBD, bibliothèques front-end, style de service web) et le **comparatif justifié** des alternatives évaluées sont détaillés à l'annexe D.

Technologie	Pourquoi	Où
MVC	Séparer les responsabilités et faciliter la maintenance	Organisation générale de l'application
Servlet	Traiter les requêtes et orchestrer les traitements	Couche de contrôle
JSP	Produire les pages dynamiques	Couche de présentation
EJB	Encapsuler la logique métier et la rendre réutilisable	Couche métier
JDBC	Accéder à la base relationnelle de façon standard	Couche d'accès aux données
XML i18n	Externaliser les textes pour la traduction hors-ligne	Module d'internationalisation
ConfigZümm.in	Paramétrer le système sans recompilation	Module de configuration
OAuth	Authentifier les utilisateurs sans gérer les mots de passe	Accès au système
Google		

Technologie	Pourquoi	Où
Service web API XML	Exposer les données aux applications tierces	Interface d'interopérabilité
X.509 et SSL	Chiffrer et authentifier les échanges	Communications inter-composants
JavaScript	Enrichir l'ergonomie du front-end	Couche de présentation

8.6 Packaging et déploiement

- **Packaging** : l'application est empaquetée sous forme d'archive Java d'entreprise (WAR pour la partie web, EAR si les EJB sont regroupés), incluant les fichiers XML d'internationalisation et le fichier `ConfigZumm.ini`.
- **Serveur** : déploiement sur un serveur d'applications J2EE fournissant conteneur de servlets et conteneur d'EJB.
- **Base de données** : SGBD relationnel accédé via JDBC.
- **Sécurité** : installation des certificats X.509 et activation de SSL pour les échanges, configuration de l'accès OAuth Google.
- **Configuration** : les seuils, unités et modules actifs sont ajustés dans `ConfigZumm.ini` sans redéploiement.

La répartition physique est illustrée par le diagramme de déploiement (figure 7.3).

CHAPITRE 9

Enveloppe budgétaire et ressources

9.1 Cadre

Le projet s'inscrit dans un cadre **académique (mini-projet)**. L'enveloppe budgétaire se traduit donc principalement en **ressources humaines, logicielles et matérielles** mobilisées, sans coût financier direct.

9.2 Ressources mobilisées

Ressource	Détail
Humaines	Équipe projet étudiante (conception, développement, tests, documentation).
Logicielles	Serveur d'applications J2EE, SGBD relationnel, IDE, outils UML, chaîne de compilation.
Matérielles	Postes de développement, capteurs simulés pour le volet indicateurs.
Externes	Compte OAuth Google, certificats X.509 (auto-signés en démonstration).

Contrainte académique majeure : tout usage de générateurs de code (ChatGPT, DeepSeek et dérivés) est proscrit et sera considéré comme une fraude à l'épreuve d'examen.

CHAPITRE 10

Délais et livrables

10.1 Livrables attendus

- Application fonctionnelle (code source J2EE + front-end).
- Plan de tests et résultats d'exécution.
- Dossier de conception (diagrammes UML).
- Rapport de projet, poster et présentation orale.

10.2 Grille d'évaluation (barème)

Critère	Points
Ergonomie	4
Fonctionnalités testées (plan de tests + exécution)	4
Technologies	4
MVC	0,5
Servlet	0,5
JSP front-end	0,25
Fichier XML et internationalisation	0,5
Fichier texte de configuration	0,25
Web service client OAuth	0,5
Web service API XML	0,5
JDBC	0,5
X.509 / SSL	0,25
EJB	0,25
Conception (UC, classes, objets, séquence, activité, état, interaction/collaboration, package, composant, déploiement)	4
Documentation	4
Rapport (structure, consistance, format)	2
Poster	1
Présentation	1
Total	20

10.3 Diagrammes UML à produire

Diagrammes de : cas d'utilisation, classes, objets, séquence, activité, machine à états, interaction / collaboration, package, composant et déploiement. Le contenu attendu de chacun est détaillé au chapitre 7 portant sur la conception.

10.4 Planning prévisionnel

Le planning est exprimé en semaines relatives, avec les livrables intermédiaires associés à chaque phase.

Phase	Semaines	Livrable
Analyse et cahier des charges	S1 à S2	Cahier des charges validé
Conception (UML)	S3 à S4	Dossier de conception
Développement du noyau CRUD	S5 à S7	Modèle et gestion des entités
Tableaux de bord et alertes	S8 à S9	Calendrier, alertes et synthèses
Services web, sécurité, i18n	S10 à S11	OAuth, API, X.509 et SSL, XML
Tests et validation	S12	Plan de tests exécuté
Documentation et présentation	S13	Rapport, poster et soutenance

CHAPITRE 11

Plan de tests et validation

La stratégie de test couvre les fonctions métier, les contraintes de gestion et les exigences techniques. Chaque cas de test précise la fonction visée, les préconditions, les étapes et le résultat attendu.

11.1 Cas de tests

ID	Fonction	Étapes	Résultat attendu
TC01	CRUD ruche	Créer, lire, modifier, supprimer une ruche	Opérations conformes et persistées
TC02	Contrainte cadres	Ajouter un 11e cadre à un corps	Refus, maximum de 10 cadres
TC03	Contrainte hausses	Ajouter une 6e hausse	Refus, maximum de 5 hausses
TC04	Contrainte socle	Retirer le dernier cadre du corps	Refus, au moins 1 cadre
TC05	Planning de visite	Proposer puis approuver une visite	Statut approuvé enregistré
TC06	Rapport de visite	Remplir tous les champs et enregistrer	Rapport persisté, case exécutée
TC07	Alerte production	Dépasser le seuil de production	Alerte de collecte affichée
TC08	Alerte baisse	Simuler une chute d'effectif	Alerte d'inspection affichée
TC09	Calcul ROI	Saisir coûts et production	ROI calculé correctement
TC10	API miel	Appeler <code>getZummHoneyActualQuantity</code>	Quantité correcte retournée
TC11	Internationalisation	Basculer FR, EN puis AR	Textes traduits via le fichier XML
TC12	Configuration	Modifier un seuil dans <code>ConfigZumm.ini</code>	Nouveau seuil pris en compte
TC13	Authentification	Se connecter par OAuth Google	Accès autorisé

ID	Fonction	Étapes	Résultat attendu
TC14	Sécurité	Vérifier SSL et certificat X.509	Échanges chiffrés et validés
TC15	Export	Exporter les enregistrements	Fichiers CSV et TXT générés
TC16	Mode hors-ligne	Saisir sans réseau puis re-connecter	Synchronisation réussie

11.2 Matrice de traçabilité

La matrice relie chaque exigence à son cas d'utilisation, aux classes et tables qui la portent, à l'écran (maquette) concerné et aux cas de test qui la valident. Elle assure la traçabilité de bout en bout, du besoin jusqu'à la vérification.

Exigence	UC	Classes / Tables	Écran	Tests
Gestion des entités et contraintes	—	Ruche, Compartiment, Cadre	Fiche ruche (fig. G.5)	TC01–TC04
Planification et approbation	UC1	Planning, Agent	Calendrier (fig. G.2)	TC05
Réalisation et rapport de visite	UC2	Visite, Photo	Rapport (fig. G.4)	TC06
Tableau de bord production	UC3	Recolte, Mesure, Seuil	TB prod. (fig. G.3)	TC07, TC09
Alerte sanitaire / baisse	UC4	Mesure, Seuil	Calendrier	TC08
Service web API (miel)	UC5	Ruche, Recolte	—	TC10
Internationalisation	—	(i18n XML)	Tous	TC11
Configuration par seuils	—	Seuil	TB prod.	TC12
Authentification OAuth	—	Agent	Connexion (fig. 7.5)	TC13
Sécurité des échanges	—	(SSL / X.509)	—	TC14
Portabilité et export	—	(export CSV/TXT)	Tous	TC15
Mode hors-ligne	UC2	Visite (file locale)	Rapport	TC16

CHAPITRE 12

Référentiel métier et bonnes pratiques apicoles

Ce chapitre précise les connaissances métier qui fondent les règles de gestion, les seuils paramétrables et les indicateurs du système. Il constitue la base fonctionnelle sur laquelle reposent les tableaux de bord et les alertes.

12.1 Facteurs de production du miel

La production d'une colonie dépend directement de la météo, de la disponibilité du pollen et du flux de nectar. Lorsque le flux de nectar est élevé, la reine pond davantage, la population culmine et le rendement atteint son maximum. En période de pénurie alimentaire, la ponte et la population diminuent. Le système doit donc corrélérer la production mesurée aux périodes et aux saisons, ce qui justifie le filtrage par mois, semaine, saison et année des tableaux de bord.

12.2 Emplacement et configuration d'un site

Le choix de l'emplacement d'un site conditionne la productivité. Les valeurs de référence suivantes peuvent alimenter les champs et les contrôles de saisie du module de gestion des sites.

Critère	Valeur de référence
Proximité des ressources	Sources de nectar et de pollen à environ 1 km.
Accès à l'eau	Eau potable accessible, plusieurs litres consommés par jour et par colonie.
Éloignement des habitations	100 m en zone forestière, 200 m en zone arbustive, 300 m en zone ouverte.
Éloignement des cultures traitées	Au moins 3 à 4 km des cultures conventionnelles utilisant des pesticides.
Surélévation et inclinaison	Ruches surélevées à environ 1,5 m du sol et légèrement inclinées pour évacuer l'humidité.
Protection	Abri contre le soleil intense, le vent et les inondations.



Figure 12.1. Exemple de ruche installée sur un site : surélévation sur pieds et planche d’envol.

12.3 Types de ruches et rendement de référence

Type de ruche	Caractéristiques et rendement
Ruche traditionnelle à rayons fixes	6 à 10 kg de miel en rayons par saison, non recommandée en apiculture biologique.
Ruche à barrettes supérieures	Largeur standard de 32 à 35 cm, inspection indépendante des rayons.
Ruche à cadres (Langstroth, Est-Africaine)	Rayons à miel réutilisables plusieurs saisons après récolte.

Ces ordres de grandeur servent de base au paramétrage des seuils de production et à l’estimation du retour sur investissement par ruche.

12.4 Classification des modèles de ruches

On distingue deux grandes familles de ruches, dont la modélisation doit rester ouverte pour couvrir les usages locaux et internationaux.

Famille	Modèles représentatifs
Ruches à cadres (verticales)	Dadant, Langstroth, Voirnot, Layens, Aumônière, Nationale (GB), WBC, Tonelli.
Ruches traditionnelles sans cadres	Warré, Kényane TBH, Tanzanienne TBH, tronc, paille.

Famille	Modèles représentatifs
Autres modèles régionaux	Lucitana, Oksman, Smith, CDB, Duvauchelle, Jarry, Congres, Zanders, modèles slovènes.

En France, les modèles les plus courants sont les ruches Dadant, Langstroth et Voirnot. Toutes les ruches à cadres partagent une composition commune, avec des dimensions précises au millimètre près, ce qui conforte le modèle de données retenu.

Ancrage du modèle : le modèle de référence du projet est la ruche Dadant, dont le corps accueille jusqu'à 10 cadres. Cette valeur fonde la capacité maximale de 10 cadres par corps ou par hausse retenue dans les règles de gestion. Le système reste toutefois paramétrable pour supporter d'autres modèles.

12.4.1 Composition commune d'une ruche à cadres

Élément	Rôle
Plancher / planche d'envol	Base de la ruche et zone d'atterrissage des abeilles.
Corps (socle)	Compartiment principal du couvain, contient les cadres de base.
Hausse	Étage d'extension destiné à la récolte du miel.
Cadre	Support amovible des rayons de cire, installé sur un slot identifié.
Grille à reine	Sépare le corps des hausses et cantonne la reine dans le corps.
Couvre-cadre et toit	Ferment et protègent la ruche des intempéries.
Nourrisseur	Permet le ravitaillement en nourriture de la colonie.

12.5 Protocole d'inspection des colonies

Les inspections sont réalisées de préférence les jours ensoleillés et portent sur des points de contrôle précis qui structurent le rapport de visite.

- développement du couvain (oeufs, larves, cellules operculées)
- remplissage des alvéoles en miel et en pollen
- présence de parasites ou de maladies
- récolte de nectar, de pollen et de propolis

Contrainte opérationnelle : une visite ne doit pas dépasser 45 minutes. Au-delà, l'agitation de la colonie augmente et se propage aux ruches voisines. Le champ *durée* du rapport de visite peut être contrôlé par rapport à cette limite de référence.

12.6 Essaimage et division des colonies

Un essaimage imminent se signale par la présence de cellules royales sur les bords des rayons. Quelques jours avant l'émergence d'une nouvelle reine, la vieille reine quitte la colonie avec la moitié des ou-

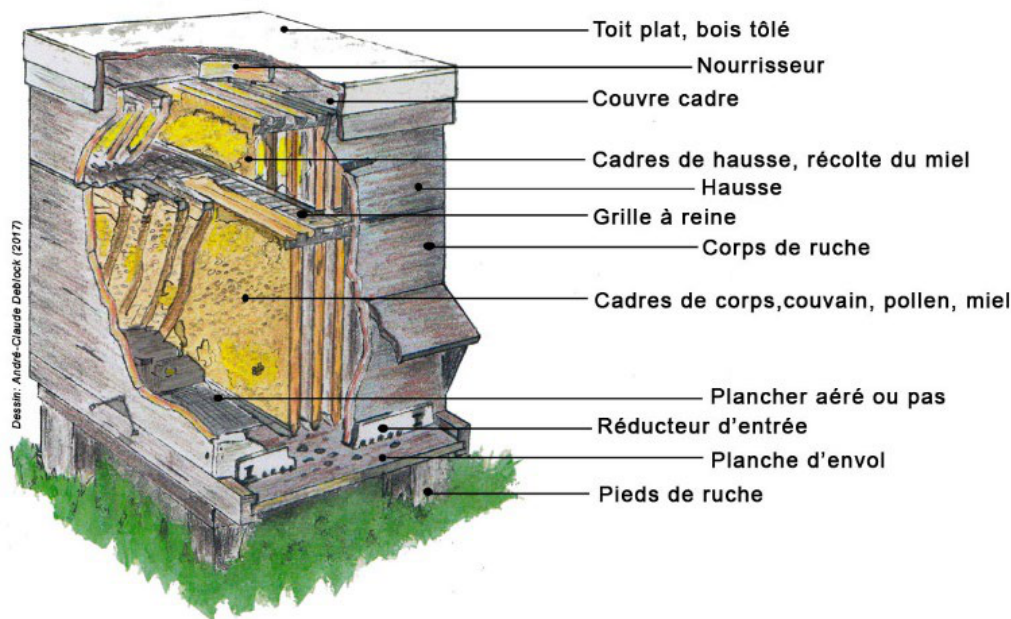


Figure 12.2. Coupe d'une ruche à cadres et rôle de chaque élément (toit, nourrisseur, couvercle, hausse, grille à reine, corps, plancher).

vrières. Le système doit donc permettre d'anticiper la division. Trois méthodes de référence sont prévues comme motifs d'intervention.

Méthode	Principe
Formation de nucléus	Retirer 1 à 2 rayons de couvain avec abeilles vers une ruche neuve.
Nucléus de reine	Transférer la reine avec un rayon sans cellules royales.
Essaimage artificiel	Déplacer la ruche mère et placer une nouvelle ruche à l'ancien emplacement avec 2 à 3 rayons de jeune couvain.

12.7 Indicateurs de bien-être et causes de baisse

Une colonie saine présente un nid à couvain bien développé aux différents stades, des alvéoles remplies de ressources, une activité de butinage visible et l'absence de parasites ou de maladies. Ces critères alimentent l'évaluation qualitative de l'état de santé et le niveau de productivité noté de 1 à 3 dans le rapport de visite.

La fuite ou l'abandon d'une ruche résulte de perturbations excessives ou de conditions inadéquates (manque de nourriture ou d'eau, températures extrêmes). Laisser une réserve de miel à la colonie lors de la récolte réduit ce risque. Ces phénomènes justifient les alertes de chute soudaine d'effectif ou de production et les seuils paramétrables associés.

12.8 Incidence sur les seuils paramétrables

Paramètre système	Ancrage métier
Seuil de collecte de miel	Rendement de référence par type de ruche et par saison.
Seuil d'alerte de baisse	Chute anormale liée à une fuite, une maladie ou une pénurie alimentaire.
Durée maximale de visite	Limite de 45 minutes par inspection.
Distances de sécurité du site	100, 200 ou 300 m selon la zone, 3 à 4 km des cultures traitées.
Périodes d'analyse	Corrélation production et saison (flux de nectar).

ANNEXE A

Annexe : Structure physique d'une ruche

Pour mémoire, une ruche à cadres amovibles se compose, de bas en haut, des éléments suivants : plancher / planche d'envol, corps (socle) contenant les cadres de couvain, grille à reine, une ou plusieurs hausses contenant les cadres de récolte du miel, couvre-cadre et toit. Cette structure justifie les règles de composition (corps obligatoire, jusqu'à 5 hausses, 1 à 10 cadres par élément) retenues dans le modèle métier.

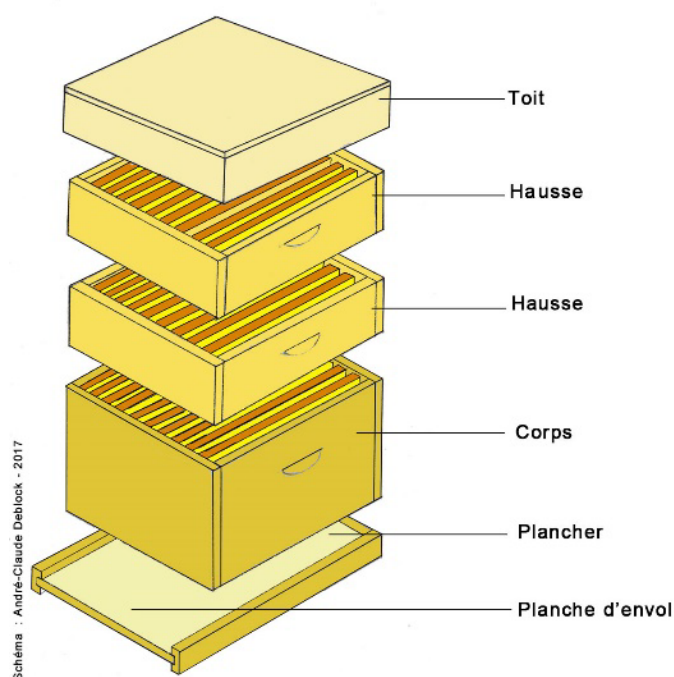


Figure A.1. Vue éclatée d'une ruche à cadres : toit, hausses, corps, plancher et planche d'envol.

ANNEXE B

Glossaire

Terme	Définition
Socle ou corps	Compartiment de base de la ruche contenant les cadres de couvain.
Hausse	Étage d'extension ajouté au corps pour la récolte du miel.
Cadre	Support amovible des rayons de cire, installé sur un slot.
Slot	Emplacement identifié d'un cadre dans un corps ou une hausse.
Essaim	Colonie d'abeilles peuplant une ruche.
Essaimage	Départ d'une partie de la colonie avec la reine pour fonder une nouvelle colonie.
Nucléus	Petite colonie constituée pour diviser un essaim ou élever une reine.
Reine	Unique femelle reproductrice de la colonie.
Couvain	Ensemble des oeufs, larves et nymphes de la colonie.
Butinage	Activité de collecte du nectar et du pollen par les abeilles.
Propolis	Substance résineuse récoltée par les abeilles.
Grille à reine	Grille qui cantonne la reine dans le corps et sépare les hausses.
Nourrisseur	Dispositif de ravitaillement en nourriture de la colonie.
Seuil paramétrable	Valeur de référence configurable déclenchant une alerte.
ROI	Retour sur investissement, rapport entre production valorisée et coûts.
CRUD	Opérations de création, lecture, mise à jour et suppression.
i18n	Internationalisation, adaptation aux langues et aux systèmes d'unités.
EJB	Composant métier de l'infrastructure J2EE.
Servlet	Composant serveur traitant les requêtes web.
JSP	Page serveur produisant l'affichage dynamique.
JDBC	Interface d'accès aux bases de données relationnelles.
OAuth	Protocole d'autorisation permettant l'authentification déléguée.
X.509 et SSL	Certificats et protocole assurant des échanges chiffrés et authentifiés.
TLS	Protocole de chiffrement des échanges réseau (couche transport), base de HTTPS.
REST / JSON	Style d'API web et format texte d'échange des données (ingestion des mesures).
MQTT	Protocole de messagerie léger (publication/abonnement) adapté aux capteurs et aux réseaux contraints.

Terme	Définition
JWT (jeton)	Jeton signé porteur d'identité, émis par le fournisseur OAuth.
RBAC	Contrôle d'accès basé sur les rôles, appliqué côté serveur (voir annexe G).
RGPD	Règlement général sur la protection des données. Encadre les données personnelles.
Rétention	Durée de conservation d'une donnée avant archivage, anonymisation ou purge.
Chiffrement au repos	Chiffrement des données stockées (colonnes sensibles) en base.
Pseudonymisation	Dissociation de l'identité d'une personne de ses données, réversible sous contrôle.
Idempotence	Propriété d'une opération rejouable sans créer de doublon (ingestion des mesures).
Hystérésis	Persistance d'un dépassement sur plusieurs mesures avant de lever une alerte (anti-rebond).

ANNEXE C

Références et sources

Le présent cahier des charges s'appuie sur les sources suivantes.

Méthode d'élaboration

- Manager-GO, *Élaborer un cahier des charges*.
<https://www.manager-go.com/gestion-de-projet/dossiers-methodes/elaborer-un-cdc>

Principes de conception logicielle

- Alex so yes, *Les principes SOLID*.
<https://alexsoyes.com/solid/>
- Refactoring Guru, *Catalogue des design patterns*.
<https://refactoring.guru>

Référentiel métier apicole

- Organic Africa, *Amélioration de la production de miel*.
<https://www.organic-africa.net/fr/agriculture-biologique/agriculture-biologique/animaux/apiculture/amelioration-de-la-production-de-miel.html>
- Au Bon Miel, *Les ruches*.
<https://www.aubonmiel.com/les-ruches/>

Fonctionnalités des solutions du marché

- HiveTracks, application de gestion apicole.
<https://www.hivetracks.com/>
- Apiary Book, solution de gestion de rucher.
<https://www.apiarybook.com/>
- BeePlus, gestionnaire d'apiculture (App Store).

Surveillance connectée (precision beekeeping)

- Arnia, surveillance acoustique et pesée des ruches.
<https://www.arnia.co/>

- BroodMinder, capteurs de température, d'humidité et de poids.

<https://broodminder.com/>

- BeeHero, plateforme IoT et analyse par intelligence artificielle.

<https://www.beehero.io/>

Retours d'expérience et limites des solutions

- Beekeeping Diary, *Best Beekeeping Apps 2026 (comparatif)*.

<https://beekeeping-diary.eu/blog/en/best-beekeeping-apps-2026/>

- HiveBook, *Free HiveTracks Alternatives*.

<https://hivebook.app/blog/hivetracks-alternatives-free>

- Beesource Forums, *Problems with Hive Tracks*.

<https://www.beesource.com/threads/problems-with-hive-tracks.249656/>

- ApiTech World, *Smart Hive Monitoring, a Beekeeper's Reality Check (2026)*.

<https://apitechworld.com/smart-hive-monitoring-a-beekeepers-reality-check-2026-review/>

ANNEXE D

Annexe : Pile technologique et comparatif des alternatives

Le socle technique de **Zümm** (MVC, Servlet, JSP, EJB, JDBC, i18n XML, OAuth Google, X.509 / SSL, service web XML) est **imposé** par le cahier des charges et par la grille d'évaluation. La liberté de choix porte donc sur les **implémentations concrètes** de ce socle : serveur d'applications, SGBD, bibliothèques front-end, style de service web. Cette annexe recense les technologies retenues couche par couche, puis justifie chaque choix face à ses alternatives.

D.1 Pile technologique retenue

Le tableau ci-dessous précise, pour chaque couche de l'architecture multi-niveaux (figure 7.2), l'implémentation concrète retenue et sa version de référence. Toutes les briques sont **libres et gratuites**, conformément à l'exigence d'autodéploiement sans abonnement.

Couche / besoin	Technologie retenue	Rôle dans Zümm
Plateforme	Jakarta EE 10 sur JDK 17 LTS	Socle « J2EE » actuel : Servlet, JSP, EJB, JAX-WS
Serveur d'applis	WildFly (conteneur Web + EJB)	Exécute l'archive WAR/EAR, fournit les conteneurs
Contrôle (MVC)	Servlets Jakarta + JSP + JSTL	Aiguillage des requêtes, couche de contrôle
Métier	EJB Session (stateless)	Règles de gestion : ROI, seuils, quantité de miel
Accès données SGBD	JDBC + pool HikariCP PostgreSQL (+ PostGIS)	Requêtes SQL, gestion des connexions Persistance relationnelle, géolocalisation des sites
Présentation	HTML5 + JavaScript + Bootstrap 5	IHM responsive, parité web / mobile
Graphiques	Chart.js	Tableaux de bord production, alertes, synthèse
Cartographie	Leaflet + OpenStreetMap	Carte des sites et rayons de butinage
i18n	Fichiers XML + ResourceBundle	Textes FR / EN / AR, support du RTL arabe

Couche / besoin	Technologie retenue	Rôle dans Zümm
Configuration	ConfigZümm.ini via Singleton	Seuils, unités et modules sans redéploiement
Service tiers	web JAX-WS (SOAP / XML)	getZümmHoneyActualQuantity(int)
Authentification	OAuth 2.0 Google	Connexion déléguée sans gestion de mots de passe
Sécurité échanges	TLS / SSL + certificats X.509	Chiffrement et authentification inter-composants
Hors-ligne	PWA (Service Worker + IndexedDB)	Saisie terrain sans réseau, synchronisation différée
Build & tests	Maven, JUnit 5, Mockito, Arquillian	Compilation, tests unitaires et in-container

Note : le passage de l'appellation historique « J2EE » à **Jakarta EE 10** ne change ni les concepts ni les API notées (Servlet, JSP, EJB, JDBC). Il ne fait qu'adopter la version maintenue de la plateforme. La trajectoire de modernisation vers Spring Boot est traitée séparément à l'annexe E.

D.2 Comparatif justifié des alternatives

Pour chaque décision ouverte, les options crédibles ont été évaluées. Le tableau retient l'option, cite les alternatives écartées et donne la justification au regard des exigences du projet (autodéploiement, gratuité, ergonomie, géolocalisation, interopérabilité).

D.2.1 Serveur d'applications

Option	Alternatives écartées	Justification du choix
WildFly	GlassFish, Payara, Open Liberty, Apache TomEE	Conteneur EJB + Servlet complet et gratuit, démarrage rapide, documentation abondante. TomEE reste plus léger mais moins intégré. Payara convient aussi (fork de GlassFish orienté production).

D.2.2 Système de gestion de base de données

Option	Alternatives écartées	Justification du choix
PostgreSQL	MySQL / MariaDB, H2, Oracle XE	Géolocalisation native (PostGIS) pour les sites et rayons de butinage, bon support des contraintes CHECK (règles cadres/hausses), séries temporelles de mesures. Gratuit et sans verrouillage propriétaire, contrairement à Oracle.

D.2.3 Style du service web tiers

Option	Alternatives écartées	Justification du choix
JAX-WS (SOAP XML)	JAX-RS (REST/JSON), / gRPC	Le cahier des charges impose un service web XML exposant <code>float getZummHoneyActualQuantity(int zummID)</code> . SOAP en est la traduction littérale, avec contrat WSDL interrogeable depuis Excel. Un endpoint REST/JSON pourra compléter l'offre en modernisation (annexe E).

D.2.4 Cartographie et rayons de butinage

Option	Alternatives écartées	Justification du choix
Leaflet OSM	+ Google Maps JS API, Mapbox	Libre et sans clé payante ni quota facturé, cohérent avec l'exigence d'autodéploiement. Tracé simple des cercles de butinage (1, 2, 3 km) à partir des coordonnées des sites.

D.2.5 Bibliothèque de graphiques

Option	Alternatives écartées	Justification du choix
Chart.js	D3.js, ApexCharts, Highcharts	Courbe d'apprentissage faible et rendu direct des histogrammes et courbes des tableaux de bord (production, ROI). D3 est plus puissant mais surdimensionné. Highcharts n'est pas libre pour un usage commercial.

D.2.6 Accès hors-ligne et mobile

Option	Alternatives écartées	Justification du choix
PWA (Service Worker + IndexedDB)	Application native (Android/iOS), Flutter, React Native	Parité web / mobile avec une seule base de code, saisie terrain sans réseau puis synchronisation différée. Évite le développement et la distribution d'applications natives, hors périmètre académique.

Cohérence avec la contrainte académique : toutes les briques retenues sont libres, documentées et *justifiables* « pourquoi / comment / où » (critère *Technologies* de la grille d'évaluation, chapitre des délais). Aucune ne repose sur un générateur de code : elles sont mises en œuvre à la main par l'équipe, conformément à la contrainte de l'épreuve.

ANNEXE E

Annexe - Choix technologique : academique et vision marche

Cette annexe repond a la question du *pourquoi* et du *comment* du langage de developpement. Elle confronte le socle **J2EE** impose par le present cahier des charges a l'ecosysteme **Spring Boot**, devenu le standard du developpement Java d'entreprise, et documente une trajectoire de modernisation sans changement de langage.

E.1 Positionnement

Le systeme **Zümm** est implemente en **J2EE (Java EE)** conformement aux exigences techniques (chapitre 7 et grille d'evaluation) : MVC, Servlets, JSP, EJB, JDBC, internationalisation externalisee, services web securises (OAuth, X.509 / SSL) et API interrogeable par des applications tierces. Ce socle materialise les competences fondamentales du parcours DSI : maitrise des couches d'un systeme d'information, separation des responsabilites et interoperabilite.

Constat marche : les briques Java EE historiques (JSP, Servlets « nus » et EJB) sont aujourd'hui considerees comme *heritees (legacy)*. Les recrutements sur les nouveaux systemes d'information privilegient tres majoritairement l'ecosysteme **Spring Boot**. La presente annexe conserve le langage **Java** et demontre, techno par techno, l'equivalent moderne de chaque exigence.

E.2 Tableau de correspondance : coeur technique

Le tableau suivant met en regard chaque technologie imposee (et notee) avec son equivalent Spring Boot, ainsi que la nature concrete du changement.

J2EE (impose)	Spring Boot	Ce qui change concretement
MVC (manuel)	Spring MVC (@Controller, @RequestMapping)	Le pattern reste, le routage devient declaratif par annotations au lieu du web.xml.
Servlet (HttpServletRequest, doGet/doPost)	@RestController / @Controller	Plus de extends HttpServlet : une methode annotee = un endpoint. Le DispatcherServlet orchestre en arriere-plan.

J2EE (impose)	Spring Boot	Ce qui change concrètement
JSP (presentation)	Thymeleaf (server-side) ou React / Angular (SPA)	Templating moderne sans scriptlets Java. Pour un effet marche fort, API REST + front decouple.
EJB (@Stateless, conteneur EJB)	Spring @Service / @Component	Meme role (logique metier, injection, transactions) sans conteneur EJB lourd. @Transactional gere les transactions.
JDBC (SQL manuel, ResultSet)	Spring Data JPA / Hibernate	Le mapping objet-relationnel remplace le SQL a la main. JpaRepository genere les CRUD (JdbcTemplate reste disponible).
i18n par fichier XML ConfigZümm.ini	messages_fr/en/ar.properties + MessageSource application.yml + @ConfigurationProperties	Principe d'externalisation des textes, format .properties standard, toujours FR / EN / AR. Parametrage sans recompilation, injectable par profil. Les seuils et unites deviennent des beans types.
OAuth Google (WS XML maison)	Spring Security OAuth2 Client	Login Google en quelques lignes de configuration au lieu d'un client OAuth ecrit a la main, et plus sur.
Web service API XML getZümmHoneyActualQuantity	API REST JSON (@GetMapping) + Swagger	JSON au lieu de XML, documentation auto-generee. XML reste possible via produces = APPLICATION_XML.
X.509 / SSL (manuel)	Spring Boot SSL (server.ssl.*) + Keystore	Activation HTTPS declarative, memes certificats X.509, configuration triviale.
JavaScript (front libre)	React / Vue / Angular (ou Thymeleaf + JS)	Front decouple consommant l'API REST, parite web et mobile simplifiee.

E.3 Tableau de correspondance : infrastructure et deployment

J2EE	Spring Boot	Benefice
Serveur d'applications lourd (WildFly, GlassFish, WebLogic)	Tomcat / Jetty embarque	L'application se lance seule : <code>java -jar app.jar</code> , plus de serveur a installer.
Packaging WAR / EAR	JAR executable (fat-jar)	Un seul artefact autonome, ideal pour le cloud et Docker.
Deployment manuel sur conteneur	Docker + docker-compose	Alignement DevOps du marche, integration continue facilitee.
SGBD relationnel via JDBC	PostgreSQL / MySQL via JPA + Flyway	Schema versionne, reproductible entre environnements.

J2EE	Spring Boot	Benefice
Gestion des dependances (Ant)	Maven / Gradle + Spring Boot Starters	Une dependance = un starter (-web, -security, -data-jpa).

E.4 Correspondance des couches

La vue en couches faiblement couplees exiguee au chapitre 7 (figure 7.2) est **integralement preservee** : aucune couche ne disparaît, seule la mise en oeuvre est modernisee.

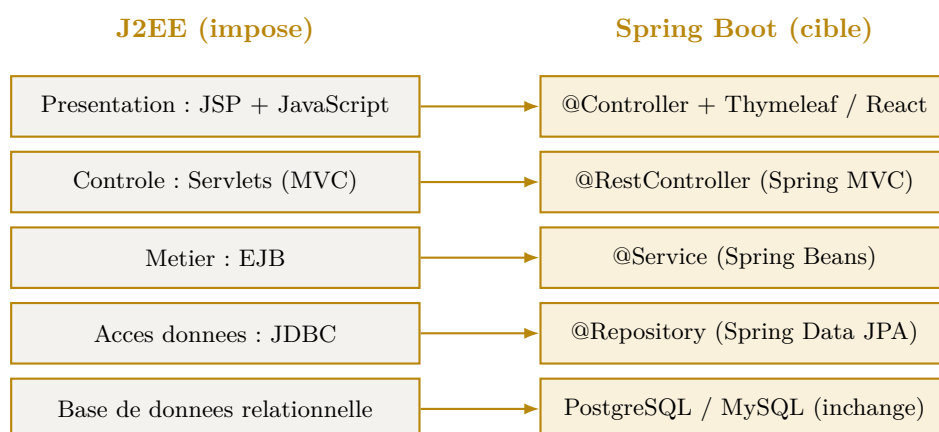


Figure E.1. Correspondance des couches J2EE vers Spring Boot : architecture preservee, mise en oeuvre modernisee.

Les modules transverses suivent la meme logique : i18n XML → MessageSource, ConfigZümm.ini → application.yml, service web XML → API REST + OpenAPI.

E.5 Argumentaire : conformite academique et vision marche

Zümm repose, conformement au cahier des charges, sur une architecture **J2EE multi-niveaux, evolutive et faiblement couplee** : MVC, Servlets, JSP, EJB, JDBC, internationalisation par fichier externalise, services web securises (OAuth, X.509 / SSL) et API interrogeable par des applications tierces. Ce socle a ete retenu et implemente **tel qu'exige**, car il constitue le referentiel d'evaluation du projet et materialise les competences fondamentales du parcours DSI : maitrise des couches d'un systeme d'information, separation des responsabilites et interoperabilite.

Une analyse **orientee marche** conduit toutefois a un constat important. Les briques Java EE historiques (JSP, Servlets « nus » et EJB) sont aujourd'hui considerees comme *heritees*. Les recrutements sur les nouveaux systemes d'information privilegient tres majoritairement l'ecosysteme **Spring Boot**, devenu le standard *de facto* du developpement Java d'entreprise. Ignorer cette realite reviendrait a livrer une solution techniquement conforme mais **decalee des pratiques professionnelles actuelles**.

Nous avons donc adopte une **demarche a deux niveaux de lecture**, sans jamais sortir du langage **Java**, ce qui garantit la continuite des competences acquises :

1. **Niveau conformite** : le livrable respecte integralement les technologies imposees et la grille d'evaluation. Les diagrammes de conception (sequence Agent → JSP → Servlet → EJB → JDBC,

couches, deploiement) refletent fidelement cette architecture.

2. **Niveau modernisation** : pour chaque technologie imposee, son equivalent Spring Boot est demontre (Servlet → @RestController, EJB → @Service, JDBC → Spring Data JPA, XML i18n → MessageSource, ConfigZumm.ini → application.yml, WS XML → API REST + OpenAPI, X.509 / SSL → configuration SSL declarative). Cette correspondance prouve que **l'architecture cible reste identique** : seule la mise en oeuvre gagne en concision, en securite et en maintenabilite.

Ce choix presente un troisieme avantage, en phase directe avec les debouches du parcours DSI. Le cahier des charges prevoit un **volet capteurs et analyse predictive** (detection d'essaimage, traitement acoustique). Ce module se prete naturellement a un **microservice Python** (pandas, scikit-learn) expose au back-end Java via REST, illustrant concretement le metier d'**analyste de donnees** cite au plan d'etudes et l'ouverture d'une **architecture faiblement couplee** a l'heterogeneite technologique.

Conclusion : le projet satisfait pleinement les exigences academiques (J2EE) tout en documentant sa **trajectoire de modernisation** (Spring Boot et microservice data). Cette double lecture transforme une contrainte pedagogique en atout professionnel : elle demontre non seulement la maitrise des fondamentaux du systeme d'information, mais aussi la capacite a faire evoluer une architecture vers les standards recherches sur le marche de l'emploi en 2026.

ANNEXE F

Annexe - Principes SOLID et design patterns

Cette annexe répond à la question du *comment* de la qualité logicielle. Elle documente l'application des cinq principes **SOLID** et d'un ensemble de **design patterns** (patrons de conception) au système **Zümm**, en mettant chaque décision en regard de la conception initiale (*avant*) et de la conception refactorisée (*après*). L'objectif est une architecture **multi-niveaux, évolutive et faiblement couplée**, conforme aux exigences du chapitre 7.

Sources méthodologiques : les définitions retenues suivent la présentation des principes SOLID de *alexsoyes.com* et le catalogue de patrons de *refactoring.guru*, adaptés au domaine apicole du projet.

F.1 Les cinq principes SOLID

Principe	Avant (conception initiale)	Après (refactorisée)
S : Responsabilité unique	L'entité <code>Ruche</code> porte la méthode de calcul <code>getZummHoneyActualQuantity()</code> : elle mélange l'état (données) et la logique métier. <code>Visite</code> regroupe données et champs de rapport.	Le calcul est extrait dans <code>ServiceMielImpl</code> (via le Composite) et le rapport devient <code>RapportVisite</code> . Chaque classe n'a plus qu'une seule raison de changer.
O : Ouvert / ferme	La détection d'alertes et la conversion d'unités reposaient sur des <code>if</code> / <code>switch</code> sur <code>typeSeuil</code> et <code>unite</code> : ajouter une règle ou une unité impose de modifier le code existant.	<code>RegleAlerte</code> (Strategy) et <code>ConvertisseurUnite</code> : une nouvelle règle ou unité = une nouvelle classe, sans toucher au <code>GestionnaireAlertes</code> . Ouvert à l'extension, ferme à la modification.
L : Substitution de Liskov	Le diagramme de cas d'utilisation généralise l'agent superviseur à partir de l'agent apiculteur. Transpose tel quel en héritage de classes, un sous-type restrictif casserait la substituable.	<code>Corps</code> et <code>Hausse</code> héritent de <code>Compartiment</code> en respectant intégralement son contrat (capacité 1..10, <code>getPoidsMiel</code>). Le rôle de l'agent passe par l'énumération <code>Rôle</code> + permissions (composition) plutôt qu'un héritage restrictif.

Principe	Avant (conception initiale)	Après (refactoree)
I : Segregation des interfaces	Un service unique « fourre-tout » (CRUD + visites + tableaux de bord + API miel) obligerait l'application tierce a dependre de methodes inutilisees.	Interfaces cibles : <code>ServiceRequeteMiel</code> (l'API tierce ne voit que <code>getZummHoneyActualQuantity</code>), <code>ServiceVisite</code> , <code>ServiceTableauDeBord</code> . Chaque client ne depend que de son contrat.
D : Inversion de dependance	La couche metier (EJB) appelle directement JDBC (implementation concrete) : couplage fort a la base et aux <code>ResultSet</code> .	La couche metier depend d'abstractions <code>RucheRepository</code> / <code>VisiteRepository</code> (pattern DAO), implementees par <code>RucheRepositoryJdbc</code> . Tests par mock et migration JDBC → Spring Data JPA facilites (cf. annexe E).

F.2 Design patterns retenus

Le tableau suivant recense les patrons appliques, leur categorie (creationnel, structurel, comportemental) et la couche concerne, avec la justification *avant / apres*.

Pattern	Avant	Après
Composite <i>structurel : metier</i>	Le calcul du miel parcourt manuellement les collections de cadres avec du code de sommation duplique.	<code>ElementRuche.getPoidsMiel()</code> est resolu recursivement (Ruche → Compartiment → Cadre) : traitement uniforme de l'arborescence.
State <i>comportemental : metier</i>	Le statut de la ruche serait teste par des <code>switch</code> disperses (creee, active, en collecte, cloturee, ...).	<code>EtatRuche</code> et ses etats concrets encapsulent les transitions autorisees. Aligement direct sur le diagramme de machine a etats.
Strategy <i>comportemental : metier</i>	Les regles d'alerte, la conversion d'unites et le ROI seraient codes en dur.	Familles d'algorithmes interchangeables (<code>RegleAlerte</code> , <code>ConvertisseurUnite</code>), injectables et testables isolement.
Observer <i>comportemental : metier</i>	Le code de mesure appellerait directement les tableaux de bord (couplage fort).	<code>GestionnaireAlertes</code> notifie les <code>Observateur</code> abonnes (tableau de bord d'alertes, notification du responsable) au franchissement d'un seuil.
Command <i>comportemental : controle</i>	Les operations terrain (collecte, divison, changement de reine) sont des appels directs, non rejouables hors-ligne.	Chaque operation devient une <code>CommandeVisite</code> . La <code>FileCommandesHorsLigne</code> rejoue les commandes a la reconnexion (mode hors-ligne + synchronisation differee).
Factory Method <i>creationnel : metier</i>	Les ruches sont creees par des <code>new</code> disperses avec initialisation manuelle des compartiments.	<code>FabriqueRuche.creer(modele)</code> produit une ruche preconfiguree selon le modele (Dadant, Langstroth...) avec corps et capacites coherents.

Pattern	Avant	Après
Builder <i>creationnel</i> <i>presentation</i>	Le rapport de visite, aux nombreux champs optionnels, imposerait un constructeur telescopique ou des setters epars.	ConstructeurRapport construit RapportVisite de facon fluide (constatations, actions, photos, evaluations).
Adapter <i>structurel</i> <i>donnees</i>	Les capteurs heterogenes (formats et unites differents) seraient traites au cas par cas.	AdaptateurCapteur uniformise chaque source en Mesure avec unites parametrables (volet automatise).
Facade <i>structurel</i> <i>controle</i>	Le Servlet orchestre directement tous les services metier.	FacadeApplication offre un point d'entree simplifie aux Servlets par-dessus les sous-systemes.
Singleton <i>creationnel</i> <i>transverse</i>	Le fichier ConfigZumm.ini et les libelles i18n seraient relus en plusieurs instances.	Configuration et GestionnaireI18n garantissent une instance unique partagee.
Proxy <i>structurel</i> <i>services web</i>	L'accès a l'API exposerait directement le service metier.	Un ProxySecurite controle l'authentification OAuth et le chiffrement SSL / X.509 avant de deleguer au service.

F.3 Vue de conception refactorée

La figure F.1 synthetise les principaux patrons dans un diagramme de classes de conception : le Composite (composition de la ruche et calcul du miel), le State (cycle de vie), les services segreges (ISP), la persistance par abstraction (DIP / Repository), le couple Strategy + Observer (alertes) et le trio Factory / Builder / Command.

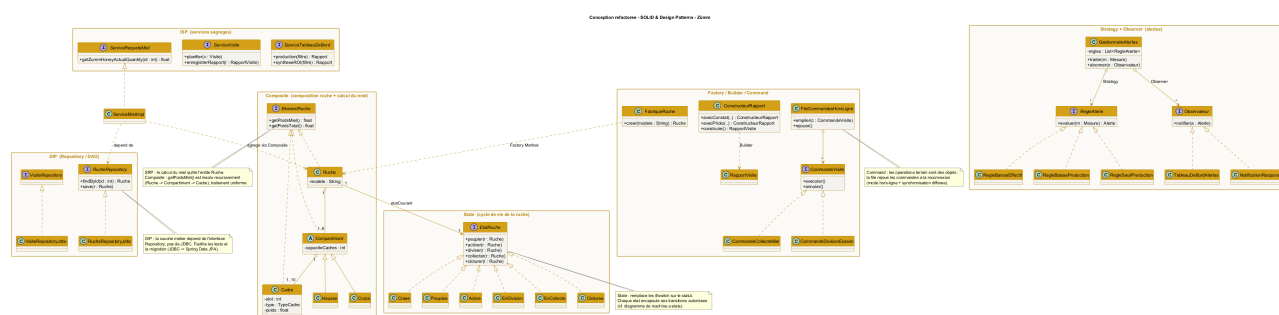


Figure F.1. Diagramme de classes refactoré : application de SOLID et des design patterns au système Zümm.

Correspondance avec les couches : les **Repository** appartiennent a la couche d'accès aux données (JDBC), les **Service** et le domaine (**Composite**, **State**, **Strategy**, **Observer**, **Command**) a la couche metier (EJB), la **Facade** et le **Proxy** a la couche de controle (Servlets), le **Builder** de rapport a la presentation (JSP). L'architecture en couches faiblement couplees du chapitre 7 est ainsi renforcee, sans etre modifiee.

F.4 Illustration dynamique : patrons Observer et Command

Deux diagrammes de sequence detaillent le comportement des patrons les plus structurants du systeme.

F.4.1 Observer (+ Strategy) : declenchement d'une alerte

La figure F.2 montre comment une mesure recue par le **GestionnaireAlertes** (sujet) est evaluee par une regle interchangeable (*Strategy*) puis, en cas de depassement, propagee aux observateurs abonnees (tableau de bord d'alertes, notification du responsable). Ajouter une regle ou un observateur n'impose aucune modification du gestionnaire (principe ouvert/ferme).

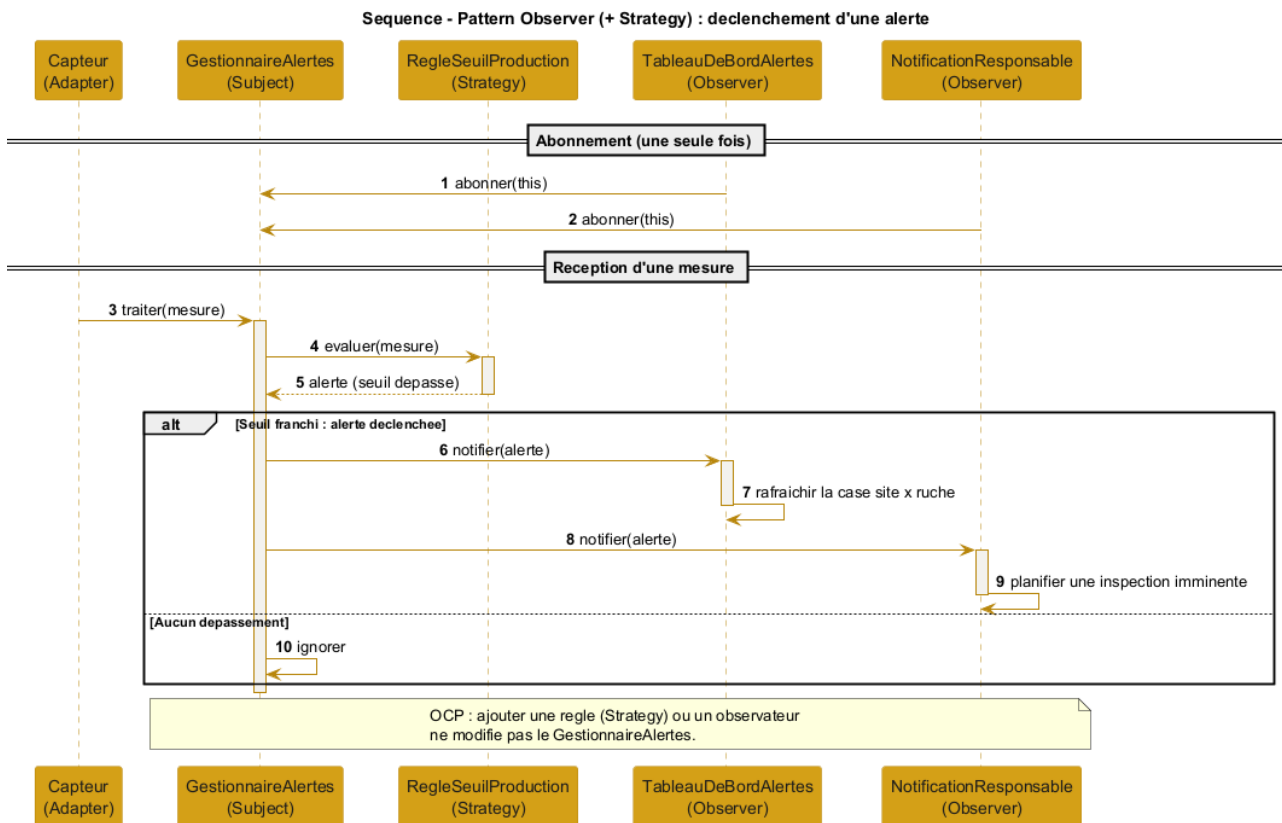


Figure F.2. Sequence du pattern Observer (+ Strategy) : flux de declenchement d'une alerte.

F.4.2 Command : mode hors-ligne et synchronisation differee

La figure F.3 illustre la saisie d'une operation terrain sans reseau : l'operation est encapsulee dans une **CommandeVisite** empilee dans la **FileCommandesHorsLigne**. Au retour du reseau, la file rejoue (**executer**) chaque commande, ou la compense (**annuler**) en cas de conflit, realisant la synchronisation differee exigee par le mode hors-ligne.

F.4.3 State : cycle de vie de la ruche

La figure F.4 montre la Ruche (contexte) qui delegue chaque operation a son etat courant. L'etat decide de la transition (activation, collecte, retour a l'etat actif) ou refuse une operation interdite dans l'etat en cours, ici un peuplement demande sur une ruche deja active. Le comportement depend ainsi de l'etat sans aucun **switch** sur le statut, en coherence avec le diagramme de machine a etats (figure 7.8).

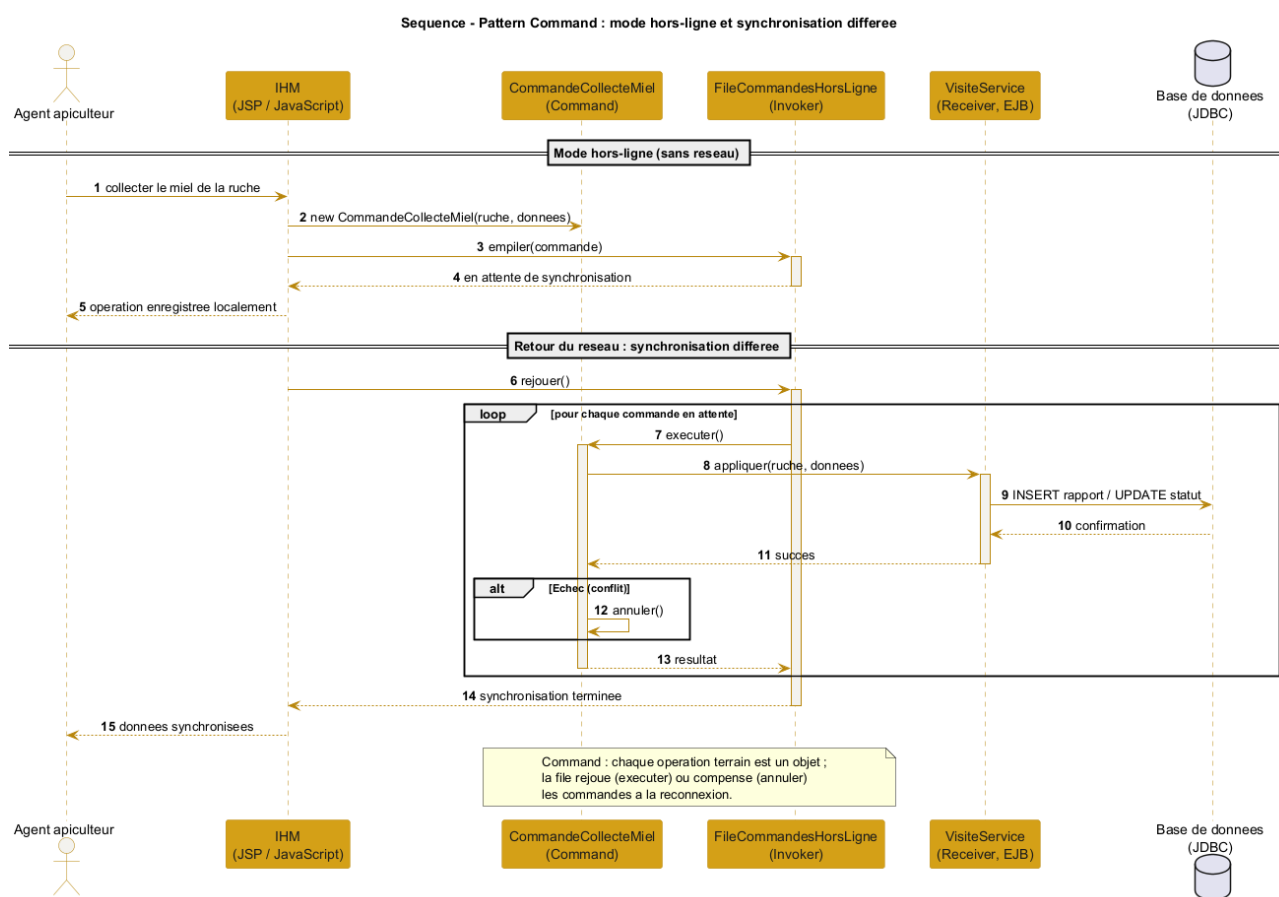


Figure F.3. Sequence du pattern Command : mode hors-ligne et synchronisation differee.

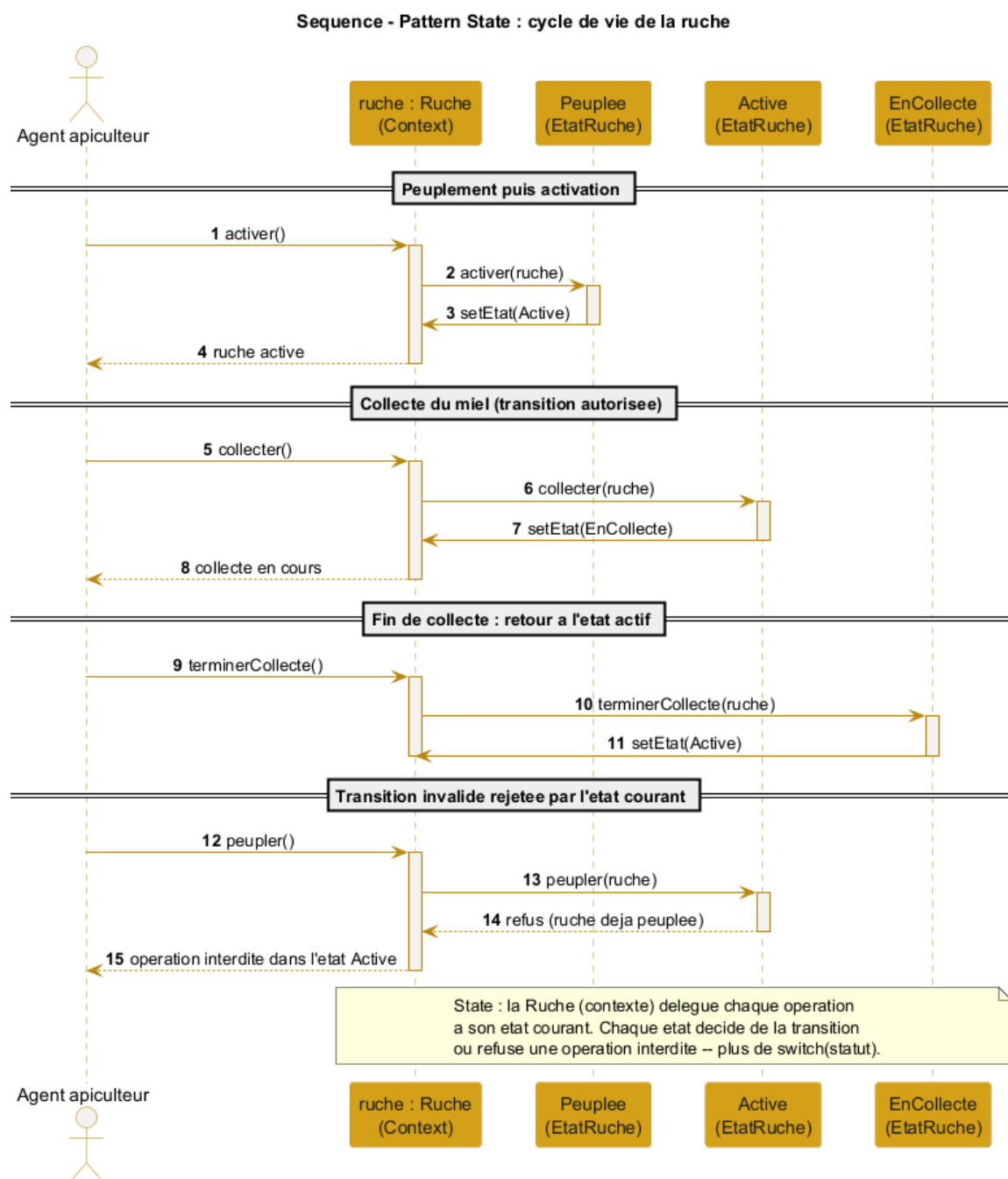


Figure F.4. Sequence du pattern State : delegation des transitions du cycle de vie de la ruche.

F.4.4 Composite : calcul de la quantité de miel

La figure F.5 détaille la résolution récursive de `getPoidsMiel()` : le client (`ServiceMielImpl`) interroge la ruche, qui propage l'appel à chaque compartiment, puis à chaque cadre (feuille). Seuls les cadres de type MIEL contribuent, et la tare est retranchée au niveau de la ruche. Le client ignore la structure interne. Ce calcul réalise la méthode `getZummHoneyActualQuantity` exposée par le service web.

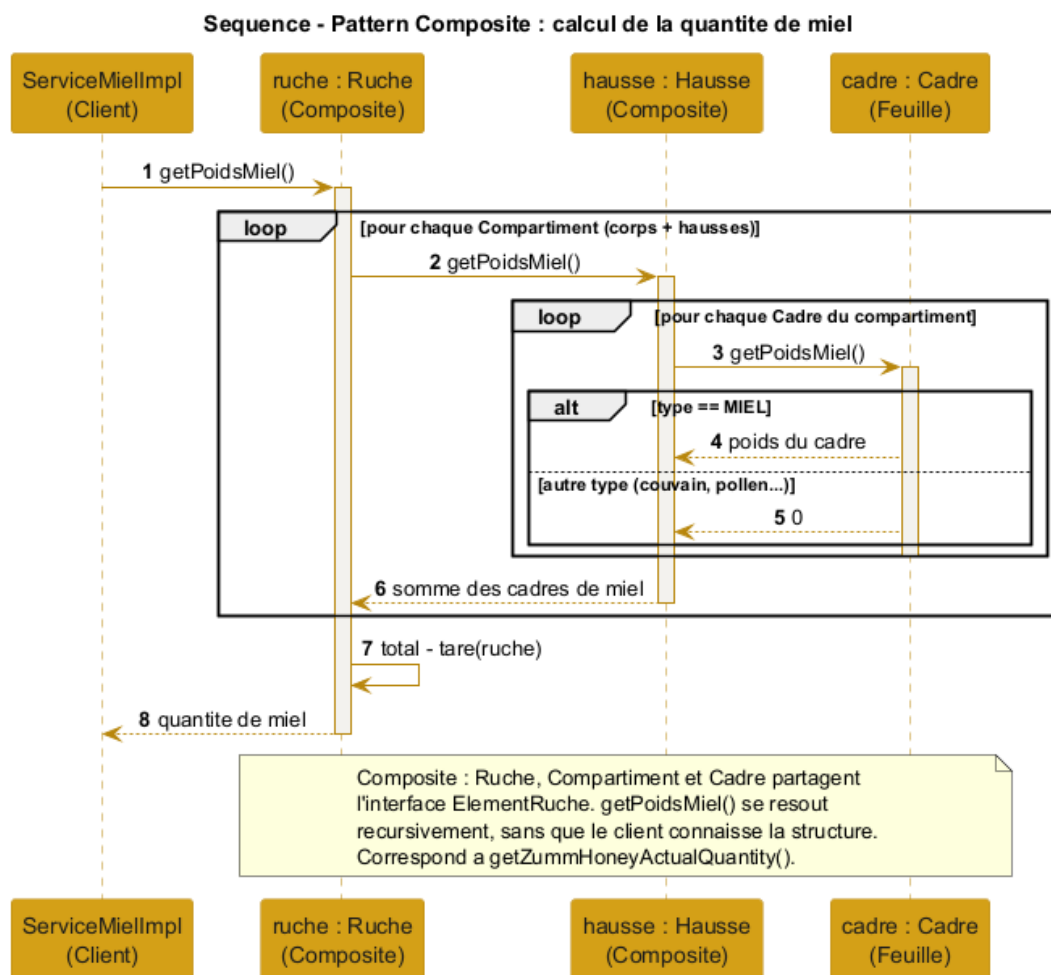


Figure F.5. Sequence du pattern Composite : calcul récursif de la quantité de miel d'une ruche.

F.4.5 Strategy : conversion d'unités hétérogènes

La figure F.6 montre la normalisation d'une mesure exprimée dans une unité quelconque : le `RegistreConvertisseur` (contexte) sélectionne la stratégie correspondant à l'unité (`ConvertisseurLbVersKg`), qui ramène la valeur à l'unité de référence avant comparaison au seuil. Une nouvelle unité se traduit par une nouvelle stratégie, sans modifier le contexte.

F.4.6 Adapter : ingestion de capteurs hétérogènes

La figure F.7 illustre l'intégration d'un capteur du marché : l'`AdaptateurBroodMinder` traduit l'API propriétaire du capteur en une `Mesure` normalisée (unités converties, horodatage, géolocalisation), exposant l'interface uniforme `Capteur` attendue par la couche métier. Les capteurs hétérogènes deviennent ainsi interchangeables.

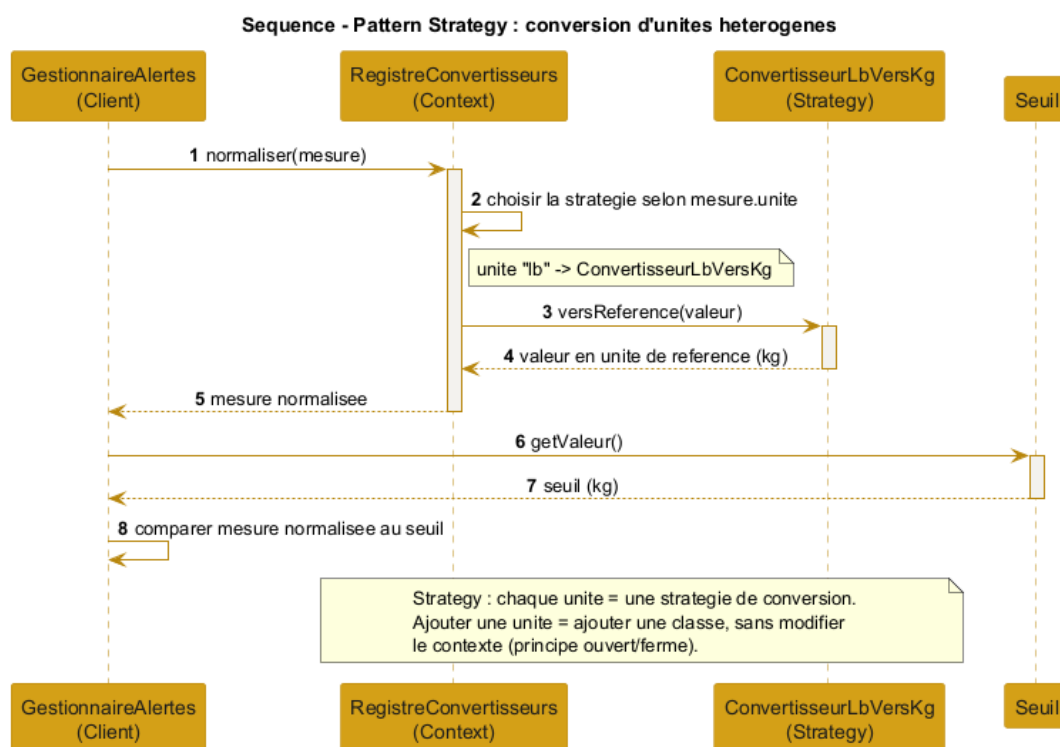


Figure F.6. Sequence du pattern Strategy : conversion d'unités hétérogènes vers l'unité de référence.

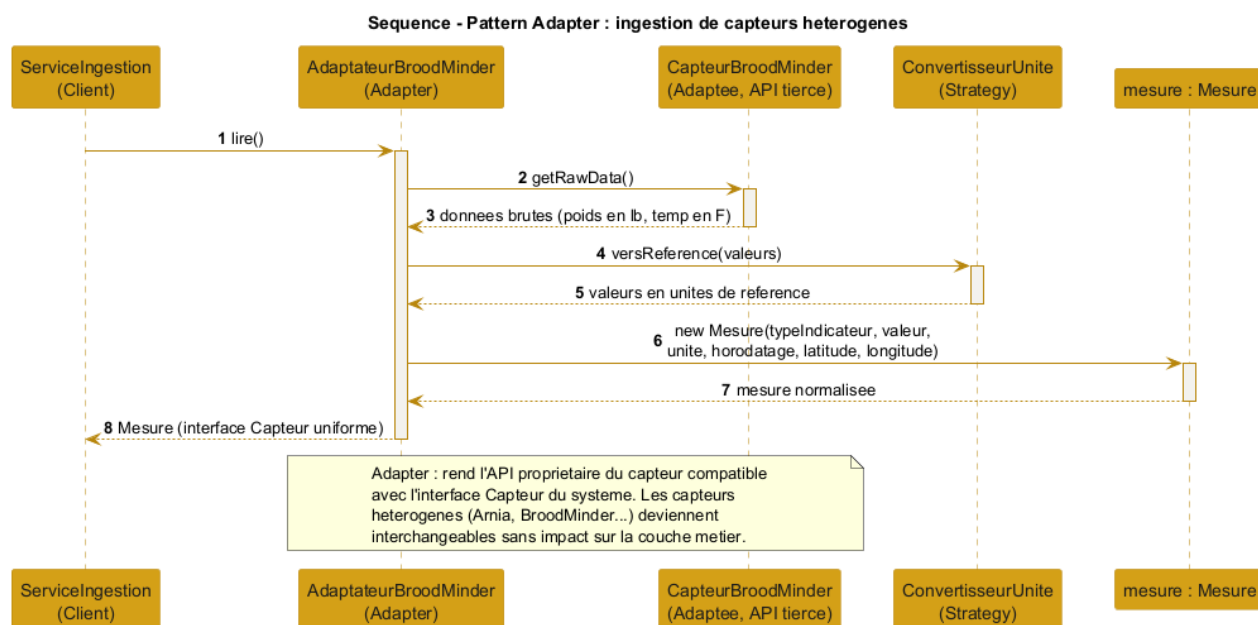


Figure F.7. Sequence du pattern Adapter : ingestion de capteurs hétérogènes (volet automatisé).

F.5 Repartition des patrons par couche

La figure F.8 synthetise la localisation de chaque patron dans l'architecture multi-niveaux. Le *Builder* se situe en presentation. La *Facade*, le *Proxy* et la file de *Command* relevent du controle. Le domaine (*Composite*, *State*, *Observer*, *Strategy*, *Factory Method*) et les services segreges (*ISP*) se placent en metier. Le *Repository* (*DAO / DIP*) et l'*Adapter* assurent l'accès aux donnees. Enfin, les *Singleton* de configuration et d'internationalisation ainsi que la *Strategy* de conversion d'unités occupent la couche transverse. Cette vue confirme que les patrons renforcent l'architecture en couches faiblement couplees sans en alterer la structure.

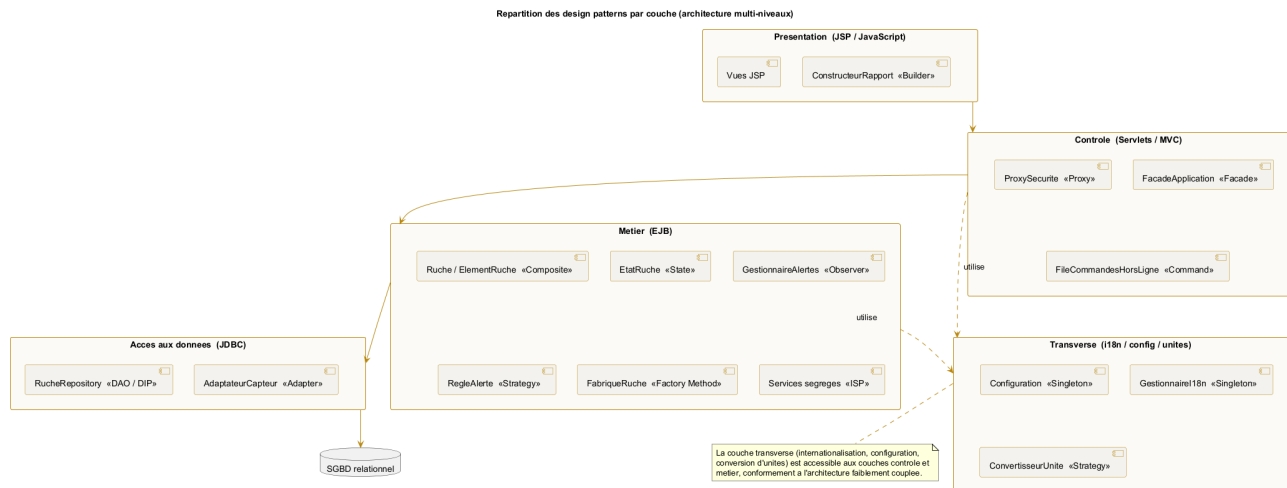


Figure F.8. Repartition des design patterns par couche de l'architecture multi-niveaux.

ANNEXE G

Annexe - Complements : donnees, IHM, securite et tests

Cette annexe regroupe des livrables complementaires renforçant le dossier : le modele logique de donnees, les maquettes de l'interface, la matrice des droits d'accès, le registre des risques, la conformite a la protection des donnees et la strategie de tests.

G.1 Modele logique de donnees (MLD)

Le MLD traduit le dictionnaire de donnees (chapitre 5) en tables relationnelles avec cles primaires, cles etrangeres et contraintes de gestion (CHECK sur le nombre de hausses, la capacite en cadres et la productivite, ainsi que l'unicite du couple compartiment/slot). Il distingue la relation d'*emplacement* (un site heberge des ruches) de la relation de *propriete* (une ferme possede des ruches), conformement a la regle autorisant un site a accueillir des ruches de plusieurs fermes.

G.1.1 Table de correspondance du vocabulaire

Pour garantir la coherence entre les livrables, chaque entite metier porte le meme concept sous trois formes : le nom du **dictionnaire de donnees** (chapitre 5), la **classe** du diagramme UML et la **table** du MLD. Convention d'ecriture des attributs : **camelCase** dans les classes (**nbHausses**), **snake_case** dans les tables (**nb_hausses**).

Table G.1. Correspondance des noms entre dictionnaire, classes et MLD.

Dictionnaire (metier)	Classe (UML)	Table (MLD)
Fermier	Fermier	FERMIER
Ferme	Ferme	FERME
Site	Site	SITE
Ruche	Ruche	RUCHE
Compartiment (corps / hausse)	Compartiment	COMPARTIMENT
Cadre	Cadre	CADRE
Agent	Agent	AGENT
Planning	Planning	PLANNING
Visite	Visite	VISITE

Dictionnaire (metier)	Classe (UML)	Table (MLD)
Photo	(rapport de visite)	PHOTO
Mesure	Mesure	MESURE
Recolte	Recolte	RECOLTE
Seuil	Seuil	SEUIL

G.2 Maquettes de l'interface (wireframes)

Les maquettes basse-fidelite suivantes fixent l'ergonomie des ecrans cles avant le developpement. Elles respectent la charte visuelle et la simplicité exigée.

G.3 Matrice des droits d'accès (RBAC)

La matrice précise, pour chaque fonction, le droit accordé à chaque profil. **Légende :** CRUD = creation / lecture / mise à jour / suppression, R = lecture, X = execution, A = approbation, — = aucun accès.

Fonction	Prop.	Apic.	Sup.	Resp.	Admin	API
Fermes et sites	CRUD	R	R	R	CRUD	—
Ruches (corps/cadres)	CRUD	R	R	R	CRUD	—
Agents	R	—	R	R	CRUD	—
Planifier une visite	R	X	X	X	X	—
Approuver le planning	—	—	A	—	X	—
Realiser visite/rapport	R	X	X	—	R	—
Calendrier agents x ruches	R	R	R	R	R	—
TB production	R	—	—	R	R	—
TB alertes / traiter	R	—	—	X	R	—
Synthese et ROI	R	—	—	R	R	—
Parametrer seuils/unites	—	—	—	R	CRUD	—
Internationalisation	—	—	—	—	CRUD	—
Export CSV / TXT	R	R	R	R	R	—
Sauvegarde / restauration	—	—	—	—	X	—
API quantite de miel	—	—	—	—	—	R

Cette matrice doit être appliquée côté serveur (vérification systématique des droits dans la couche de contrôle), et non uniquement par le masquage des éléments d'interface.

G.4 Registre des risques

Risque	Probab.	Impact	Mesure de reduction
Retard de developpement	Moyenne	Fort	Prioriser le noyau CRUD, avec jalons hebdomadaires (S5 à S12).

Risque	Probab.	Impact	Mesure de reduction
Portee trop large	Moyenne	Fort	Se limiter aux fonctions de priorite haute, le reste en option.
Complexite i18n arabe (RTL)	Moyenne	Moyen	Test precoce de bout en bout FR/EN/AR, gestion <code>dir=rtl</code> .
Perte de donnees terrain	Moyenne	Fort	File de commandes hors-ligne + synchronisation + sauvegarde.
Faibles applicatives (injection, XSS)	Moyenne	Fort	<code>PreparedStatement</code> , echappement JSP, revue de securite.
Fausse alertes (capteurs)	Moyenne	Moyen	Seuils parametrables + validation humaine par le rapport.
Indisponibilite d'OAuth Google	Faible	Moyen	Repli d'authentification locale pour la demonstration.
Volumetrie des mesures	Faible	Moyen	Archivage et agregation par periode (volet automatise).
Usage d'un generateur de code	Faible	Tres fort	Conception et code faits maison, tracabilite et revue par pairs.

G.5 Protection des donnees personnelles

Le systeme manipule des donnees personnelles (noms et contacts des fermiers et des agents) et des donnees potentiellement sensibles (**geolocalisation** precise des sites). Les principes suivants sont retenus, dans l'esprit du RGPD :

- **Minimisation et finalite** : ne collecter que les donnees utiles au suivi apicole.
- **Securite** : chiffrement des echanges (SSL / X.509) et acces authentifies, deja prevus.
- **Controle d'accès** : application stricte de la matrice RBAC ci-dessus.
- **Droits des personnes** : consultation, rectification et **effacement** des donnees, **portabilite** assuree par l'export CSV / TXT.
- **Conservation** : duree de retention definie pour les mesures et les visites, au-dela de laquelle les donnees sont archivees ou anonymisees.
- **Tracabilite** : journalisation des acces et des actions sensibles (approbation de planning, cloture de site).

Les principes ci-dessus se traduisent par les mesures techniques suivantes, par categorie de donnees. Le **chiffrement au repos** vise en priorite les colonnes sensibles (contacts, geolocalisation). La **pseudonymisation** dissocie l'identite de l'agent de l'historique agronomique lors de l'anonymisation.

Table G.4. Mesures techniques de protection des donnees (esprit RGPD).

Donnee	Finalite	Retention	Securite / effacement
Identite (nom, contact)	Comptes et responsabilites	Relation + 1 an	Chiffrement au repos, RBAC, anonymisation sur demande
Geolocalisation des sites	Suivi, cartes, meteo	Vie du site + 1 an	Chiffrement au repos, precision reducible, effacement a la cloture

Donnee	Finalite	Retention	Securite / effacement
Visites et rapports	Tracabilite sanitaire et production	5 ans	Chiffrement au repos, pseudonymisation (dissociation de l'agent)
Photos d'inspection	Suivi visuel de la colonie	2 ans	Stockage chiffré, accès restreint, suppression avec le rapport
Mesures capteurs	Analyse de performance	Brutes 1 an, puis agrégées	Chiffrement au repos, agrégation / anonymisation
Journaux d'accès	Securite et imputabilité	6 a 12 mois	Accès administrateur, purge automatique

Procédure d'effacement : sur demande d'une personne concernée, le système produit d'abord un export (portabilité CSV / TXT), puis efface ou anonymise les enregistrements selon la table ci-dessus. Un **registre des traitements** et un référent protection des données sont tenus pour le projet.

G.6 Stratégie de tests

La stratégie complète le plan de tests fonctionnels (chapitre 10) par les niveaux unitaire et d'intégration. La conception faiblement couplée (interfaces **Repository**, services séparés) rend les règles métier testables en isolation, par injection de doubles (mocks).

Niveau	Portée	Exemples	Outil
Unitaire	Règles de gestion isolées	Contraintes cadres/hausse, calcul du miel (Composite), ROI, seuils	JUnit 5, Mockito
Intégration	Accès aux données (Repository/JDBC)	CRUD persiste, requêtes de calcul	JUnit + base H2
Système	Cas fonctionnels de bout en bout	Cas TC01 à TC16 du chapitre 10	JUnit, Selenium
Acceptation	Validation par la démonstration	Parcours agent, fermier, responsable	Soutenance

- **Couverture visée** : au moins 70 % de la couche métier (règles de gestion), mesurée par JaCoCo.
- **Tracabilité** : chaque exigence est reliée à au moins un cas de test (matrice du chapitre 10), complétée par les tests unitaires correspondants.
- **Automatisation** : exécution des tests à chaque build (Maven), pré-requis à la livraison.

Synergie avec la conception : l'inversion de dépendance (DIP) permet de substituer un **RucheRepository** en mémoire aux implémentations JDBC, ce qui rend les tests rapides, déterministes et indépendants de la base.

Modele Logique de Donnees (MLD) - Zümm

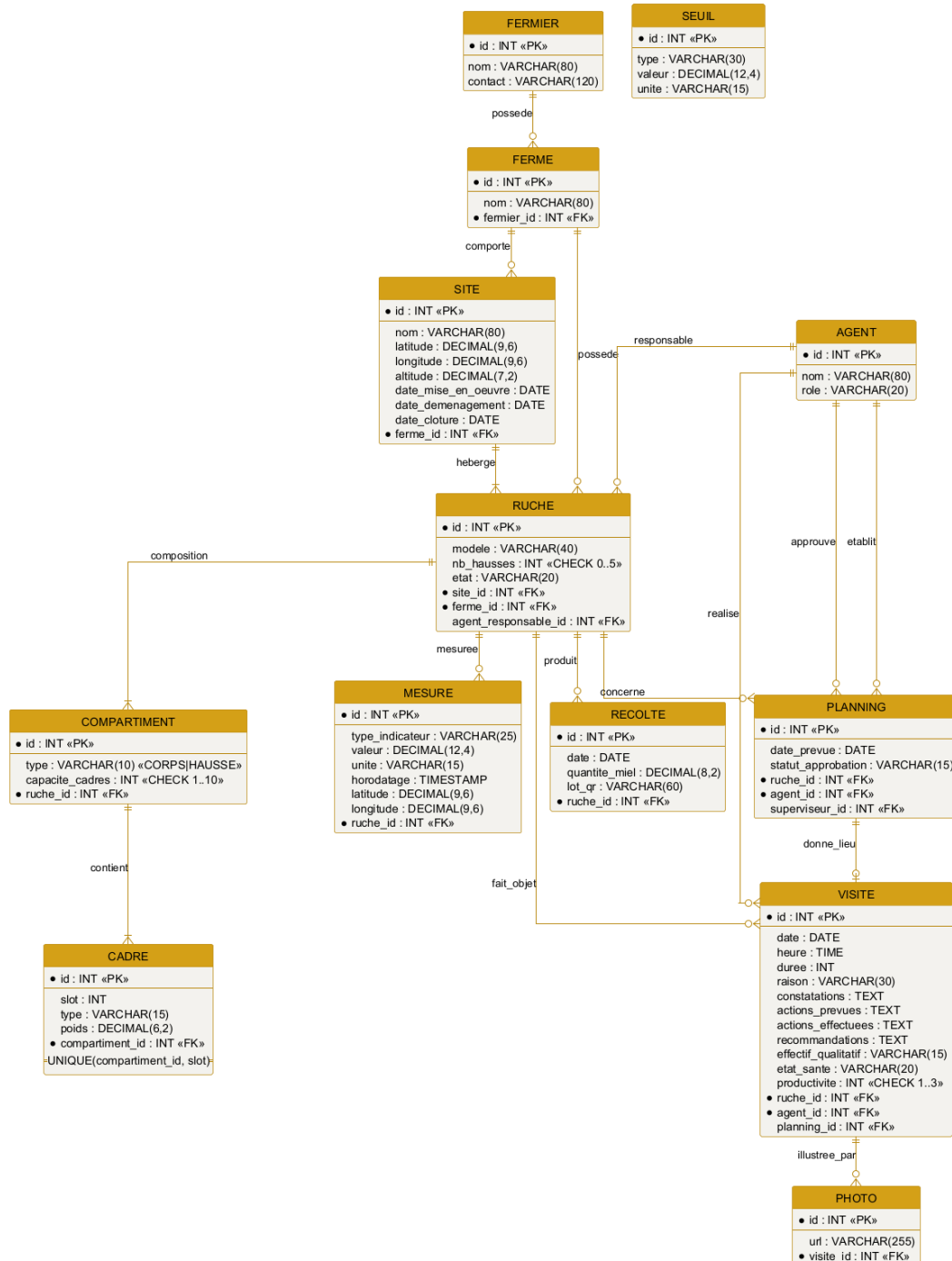


Figure G.1. Modele logique de donnees relationnel du systeme Zümm.

Zümm Calendrier Tableaux de bord Ruches Agents Deconnexion

Filtres: Site: ^Tous^ Agent: ^Tous^ Vue: ^Mois^

	Ruche R1	Ruche R2	Ruche R3
Agent A	OK executee		O prevue
Agent B		O prevue	
Agent C	O prevue		OK executee

Pop-up (clic sur une case)

Date / heure:

Raison: ▼

Actions prevues:

Actions effectuees:

Figure G.2. Maquette : calendrier / matrice agents x ruches avec pop-up de visite.

Zümm Calendrier Tableaux de bord Ruches Agents

Production ☐ Alertes ☐ Synthese / ROI ☐

Ferme: ^Toutes^ Site: ^Tous^ Periode: ^Saison^ Seuil (kg): "25"

Site	Ruche	Production (kg)	Statut
Nord	R1	28	depasse : collecte
Nord	R3	31	depasse : extension
Sud	R7	26	depasse : collecte

Production par site

Nord: R1=28 R3=31 Sud: R7=26 Seuil=25 kg

Figure G.3. Maquette : tableau de bord de production avec seuil parametrable.

Zümm Calendrier Tableaux de bord Ruches Agents

Rapport de visite - Ruche R1 (Dadant)

Date Heure Duree (min)

Raison ▼

Constatations

Actions prevues

Actions effectuees

Recommandations

Effectif ▼ Etat de sante ▼ Productivite ☒ 1 () 2 () 3

Photos

Figure G.4. Maquette : formulaire de rapport de visite (avec mode hors-ligne et photos).

Maquette d'une interface utilisateur pour la gestion des ruches. Le menu de navigation en haut contient 'Zümm', 'Calendrier', 'Tableaux de bord', 'Ruches' (sélectionné) et 'Agents'. Le titre de la section est 'Fiche ruche R1'. Les champs de formulaire incluent : 'Modele' avec une liste déroulante 'Dadant', 'Etat' avec une liste déroulante 'Active', 'Agent responsable' avec une liste déroulante 'K. Ben Ali', 'Site' avec une liste déroulante 'Rucher Nord', et 'Nb hausses' avec le chiffre '2'. La section de composition est structurée en arbre :

- Ruche R1
 - Corps (capacite 10 cadres)
 - Cadre 1 - couvain
 - Cadre 2 - couvain
 - Hausse 1 (capacite 10 cadres)
 - Cadre 1 - miel
 - Hausse 2 (capacite 10 cadres)
 - Cadre 1 - miel

En bas, il y a trois boutons : 'Ajouter une hausse', 'Ajouter un cadre' et 'Planifier une visite'.

Figure G.5. Maquette : fiche d'une ruche et composition (corps, hausses, cadres).

ANNEXE H

Annexe - Intelligence artificielle et analyse predictive

Cette annexe répond aux questions du *pourquoi*, du *comment* et du *où* de l'intelligence artificielle dans **Zümm**. Elle ne crée aucune exigence nouvelle : elle prolonge le volet capteurs et l'analyse predictive déjà anticipés au chapitre 7 (§ 4.5) et la trajectoire de microservice *data* évoquée à l'annexe E. L'objectif est de documenter une intégration de l'IA **cohérente avec l'architecture faiblement couplée, sans trahir les contraintes** du cahier des charges (autodéploiement, gratuite, validation humaine).

Positionnement : dans Zümm, l'IA est un **module d'aide à la décision**, jamais un organe de commande. Elle produit des *pre-alertes* explicables ; l'agent apiculteur conserve la **validation finale** par le rapport de visite, conformément à la règle d'évaluation du § 4.5.4.

H.1 Principes directeurs

Toute brique d'IA retenue respecte quatre principes issus directement des exigences du projet :

- **Aide à la décision et validation humaine** : l'IA signale, elle ne décide pas. Chaque sortie est confirmée par l'agent (§ 4.5.4).
- **Autodéploiement sans service tiers obligatoire** : aucune fonction essentielle ne dépend d'une API cloud payante. Les traitements de base tournent **en local** ; un recours cloud reste *optionnel* et activable par configuration.
- **Decouplage et modularité** : l'IA est un **module activable** via `ConfigZumm.ini`, isole du cœur métier par une interface, dans l'esprit du couplage faible (annexe F).
- **Explicabilité et frugalité** : on privilégie les méthodes **interprétables** (statistiques, règles) et économes, plutôt que des modèles opaques, pour rester défendables et reproductibles.

H.2 Cartographie des usages d'IA

Le tableau ci-dessous recense les usages d'IA pertinents pour Zümm, en distinguant ce qui est **retenu au périmètre** (implementable dans le socle présent) de ce qui est **anticipé** (interface spécifiée, développement ultérieur au titre du volet capteurs).

Usage	Description	Dep. cloud	Effort	Statut
Detection d'anomalie adaptative	Ligne de base par ruche remplaçant le seuil fixe, pour detecter une derive inhabituelle de production, d'effectif ou de temperature.	Non	Faible	Retenu
Detection acoustique d'essaimage	Analyse de la signature sonore pour une pre-alerte d'essaimage 1 a 5 jours a l'avance (precision beekeeping, § 4.5.1).	Non	Eleve	Anticipe
Vision : detection de maladies	Aide a l'identification (varroa, couvain) a partir des photos d'inspection attachees au rapport de visite.	Non	Eleve	Anticipe
Saisie vocale assistee	Dictee du compte rendu, transcription alimentant les champs du rapport (§ 4.7, priorite basse).	Optionnel	Moyen	Anticipe
Assistant de synthese	Resume en langage naturel des tableaux de bord (ROI, alertes) pour le proprietaire.	Optionnel	Moyen	Optionnel

Recommandation de priorisation : livrer la **detection d'anomalie adaptative** comme fonction reelle (faisable dans le socle impose, sans dependance cloud), et **specifier les autres usages comme interfaces anticipees**. C'est la posture deja adoptee par le cahier des charges pour le volet capteurs, ce qui preserve la coherence du document.

H.3 Cas retenu : detection d'anomalie adaptative

Le § 4.5.4 definit des **seuils fixes** (par exemple temperature interne $< 32^{\circ}\text{C}$ ou $> 36^{\circ}\text{C}$) et une regle d'anti-rebond. La limite d'un seuil fixe est qu'il ignore le **comportement propre de chaque ruche** et sa **saisonnalite**. L'evolution naturelle consiste a apprendre une **ligne de base par ruche** et a mesurer l'ecart a cette ligne, ce qui detecte la « baisse relative $> 20\%$ vs periode precedente » de facon adaptative.

H.3.1 Formalisation (sans code)

Pour chaque couple (ruche, indicateur), on entretient une **moyenne mobile exponentielle** (EWMA) μ_t et une estimation de dispersion σ_t , mises a jour a chaque nouvelle mesure normalisee x_t (unite de reference, § 5.3) :

$$\mu_t = \alpha x_t + (1 - \alpha) \mu_{t-1}, \quad (\text{H.1})$$

$$v_t = \alpha (x_t - \mu_{t-1})^2 + (1 - \alpha) v_{t-1}, \quad \sigma_t = \sqrt{v_t}. \quad (\text{H.2})$$

Le **score d'anomalie** est l'ecart normalise (*z-score* glissant) :

$$z_t = \frac{|x_t - \mu_{t-1}|}{\sigma_{t-1} + \varepsilon}. \quad (\text{H.3})$$

Une **pre-alerte** n'est levée que si $z_t > k$ (sensibilité) **de manière persistante sur N mesures consecutives** (hysteresis, comme au § 4.5.4). Cette continuité avec la règle existante est volontaire : on remplace un seuil absolu par un seuil *relatif appris*, sans changer la logique d'anti-rebond ni le principe de validation humaine.

H.3.2 Parametres (tous paramétrables via ConfigZümm.ini)

Parametre	Role	Effet du réglage
α (lissage)	Poids de la mesure recente	Plus α est grand, plus la ligne de base reagit vite (et plus elle est sensible au bruit).
k (sensibilité)	Seuil sur le z-score	Plus k est petit, plus la detection est sensible (et plus les fausses alertes augmentent).
N (persistance)	Nombre de mesures consecutives	Plus N est grand, moins il y a de fausses alertes (au prix d'un leger delai).
Fenetre / saison	Historique de reference	Permet une ligne de base par saison (miellee, hivernage).

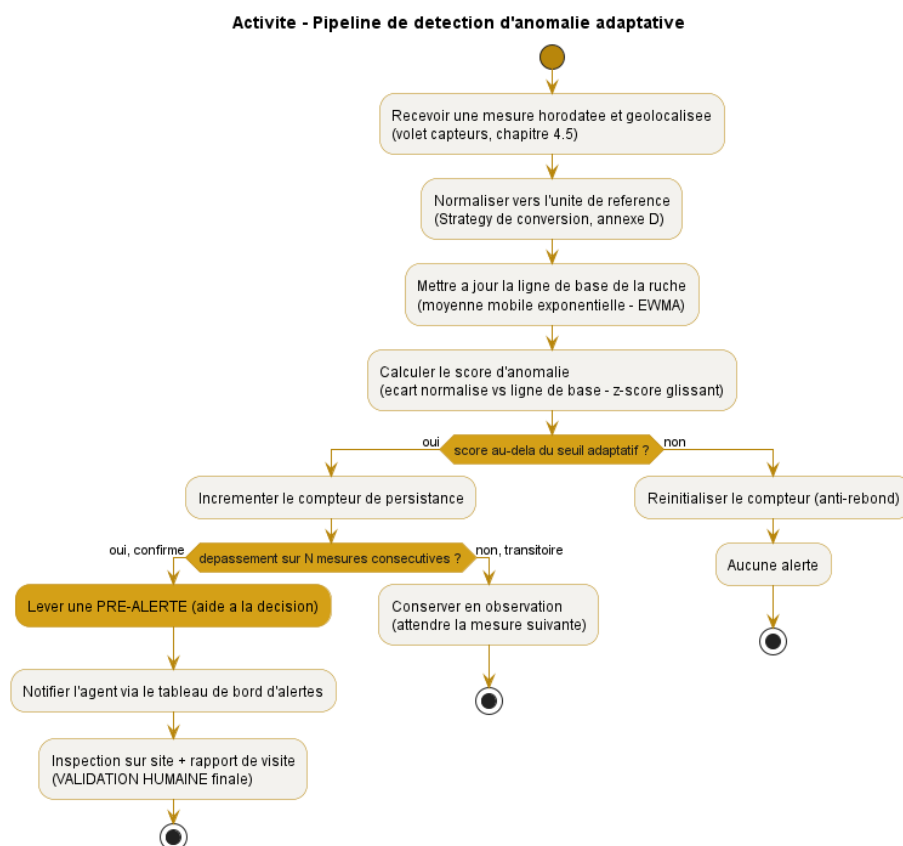


Figure H.1. Pipeline de detection d'anomalie adaptative, de la mesure brute a la pre-alerte confirmee par l'agent.

Coherence avec le § 4.5.4 : la valeur est d'abord **convertie vers l'unité de référence** (Strategy de conversion, annexe F), puis comparée à un seuil ; l'alerte reste soumise à l'**anti-rebond** et demeure une **aide à la décision**. Seule la nature du seuil change : de *fixe* à *appris par ruche*.

H.4 Architecture d'intégration

L'IA s'insère sans rien casser de l'architecture en couches (figure 7.2). Deux mécanismes de découplage sont mobilisés.

H.4.1 Le détecteur comme Strategy interchangeable

La détection d'anomalie devient une **stratégie de RegleAlerte** (pattern Strategy, annexe F), au même titre que le seuil fixe existant. Le **GestionnaireAlertes** choisit, par configuration, entre « seuil fixe » et « ligne de base adaptative » sans que son code change (principe ouvert/fermé). L'IA n'est donc pas un ajout invasif, mais une **implémentation supplémentaire d'une interface déjà prévue**.

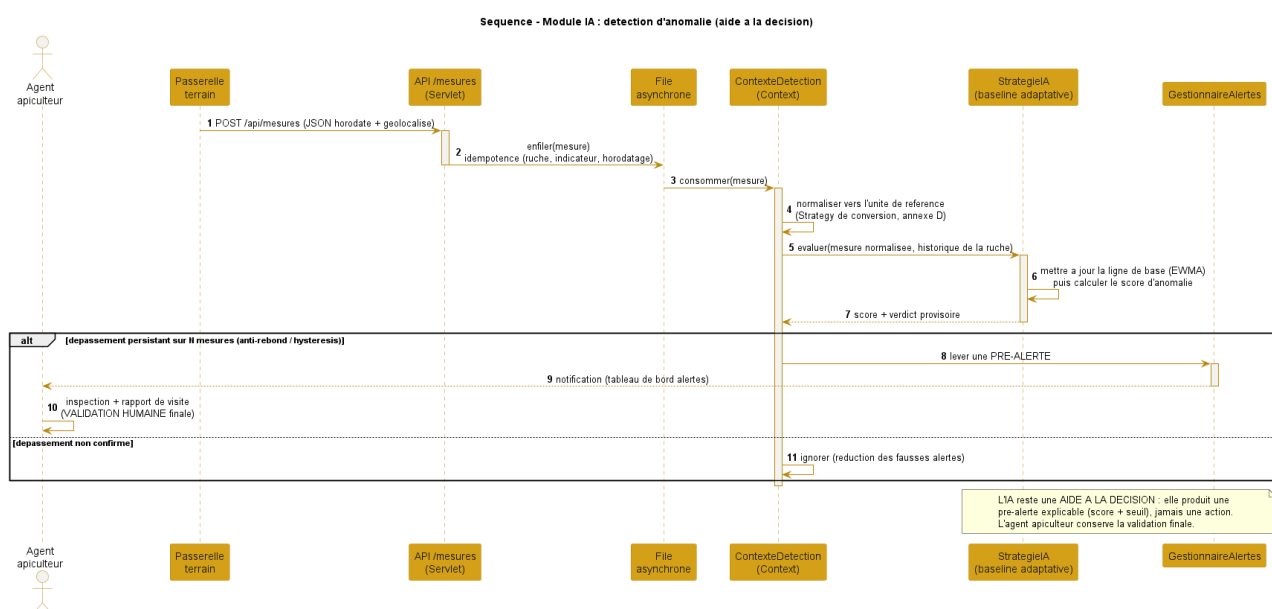


Figure H.2. Sequence du module IA : la mesure ingérée est évaluée par la stratégie adaptative, qui lève une pré-alerte confirmée par l'agent.

H.4.2 Les traitements lourds comme microservice decouplé

Les usages exigeant du **signal ou de la vision** (acoustique, photos) ne doivent **jamais** s'exécuter dans le conteneur EJB. Ils sont déportés dans un **microservice Python** (pandas, scikit-learn / TensorFlow), exposé au cœur Java par **REST/JSON**, exactement la trajectoire *data* décrite à l'annexe E. Ce microservice est **optionnel** : en son absence, le volet manuel et la détection statistique de base fonctionnent intégralement.

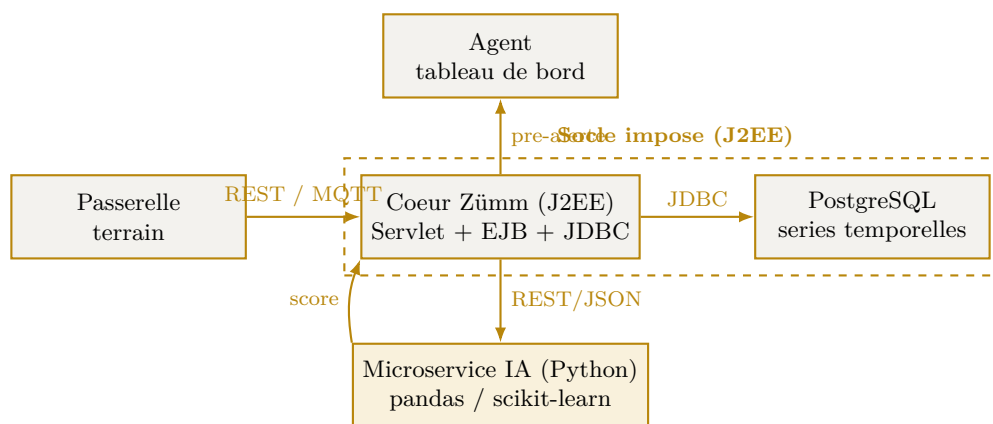


Figure H.3. Intégration découplée : le socle J2EE reste maître, le microservice IA est un module optionnel branché par service web.

H.5 Usages anticipés (interfaces spécifiées, hors périmètre de développement)

Conformément au traitement du volet capteurs, les usages avancés sont **spécifiés comme interfaces** pour que le modèle de données et l'architecture les accueillent, sans être développés dans le présent projet.

- **Acoustique / essaimage** : le modèle de données accueille déjà un indicateur `SIGNATURE_ACOUSTIQUE` (§ 4.5). Le microservice IA renvoie un verdict (motif d'essaimage détecté, présence de la reine) qui alimente une pré-alerte.
- **Vision / maladies** : les **photos d'inspection** déjà attachées au rapport de visite constituent le jeu d'entrée ; un modèle pré-entraîné renvoie une suggestion, validée par l'agent.
- **Saisie vocale** : la transcription peut s'appuyer sur la **Web Speech API du navigateur** (gratuite, côté client, sans dépendance serveur), avec repli cloud *optionnel* activé par `ConfigZümm.ini`.

H.6 Ethique, limites et conformité

Enjeu	Disposition retenue
Fausse alerte	Hysteresis sur N mesures et sensibilité k réglables ; l'alerte reste indicative.
Surconfiance dans l'automatisation	Positionnement explicite en aide à la décision ; l'agent conserve la validation finale (§ 4.5.4).
Explicabilité	Méthodes interprétables (score + seuil) préférées aux modèles opaques ; la pré-alerte affiche sa justification.
Protection des données	Traitement local par défaut, chiffrement SSL / X.509 des échanges, pas de transfert cloud imposé.
Absence de verrouillage	Le microservice IA est optionnel et remplaçable ; sans lui, la gestion et la détection de base restent opérationnelles.
Frugalité	Priorité aux traitements statistiques légers ; l'inférence lourde n'est déclenchée qu'à la demande.

Conclusion : l'IA de Zümm se resume a une regle claire — *apprendre la normale de chaque ruche, signaler l'ecart, laisser l'humain trancher*. Cette approche ajoute de la valeur predictive tout en respectant integralement les contraintes d'autodéploiement, de couplage faible et de validation humaine du cahier des charges. Elle transforme le volet predictif *anticipe* en une trajectoire d'implementation **concrete et progressive**.

ANNEXE I

Annexe - Securite et robustesse

La securite est une exigence **transverse** deja presente dans le cahier des charges : chiffrement des echanges par **X.509 / SSL**, authentication deleguee par **OAuth Google**, matrice des droits **RBAC** et conformite **RGPD** (annexe **G**). Cette annexe la **consolide** en une strategie coherente de **defense en profondeur**, puis traite la **robustesse** (resistance a la charge et aux pics), car la *disponibilite* est l'une des trois proprietes de securite. Les bonnes pratiques retenues suivent les guides de reference du domaine (durcissement de site web) et l'outillage libre **k6** pour les tests de charge.

Triade CID : la securite vise la **Confidentialite** (chiffrement, controle d'accès), l'**Integrite** (validation des entrees, requetes preparees, sauvegardes) et la **Disponibilite** (anti-DDoS, limitation de debit, tests de charge). Aucune brique n'est un service tiers *obligatoire* : toutes ont un equivalent libre et autodeployable (ModSecurity, Let's Encrypt, k6), conformement a l'exigence d'autodeploiement.

I.1 Modele de menace synthetique

Le tableau met en regard les risques usuels d'une application web avec la mesure retenue dans Zümm.

Menace	Mesure retenue dans Zümm
Interception des echanges	TLS / SSL de bout en bout, certificats X.509, redirection HTTPS et en-tete HSTS.
Vol d'identifiants	OAuth Google (pas de mot de passe stocke), double authentication portee par le fournisseur.
Injection SQL	Requetes preparees (<i>PreparedStatement</i>) via JDBC, validation et typage des entrees.
XSS (script injecte)	Echappement systematique des sorties, en-tete Content-Security-Policy.
CSRF (requete forgee)	Jetons anti-CSRF par session, cookies SameSite.
Detournement de session	Cookies HttpOnly et Secure, expiration et rotation de session (deja § 7).
Elevation de privileges	Controle d'accès RBAC et moindre privilege (annexe G).

Menace	Mesure retenue dans Zümm
Deni de service (DDoS)	Limitation de debit, pare-feu applicatif (WAF), ingestion asynchrone des mesures.
Exposition de données	Chiffrement au repos, minimisation RGPD, sauvegardes chiffrées.

I.2 Defense en profondeur

La securite ne repose pas sur une barriere unique mais sur des **couches successives** : si l'une cede, les autres tiennent. La figure I.1 projette ces couches sur l'architecture de Zümm.

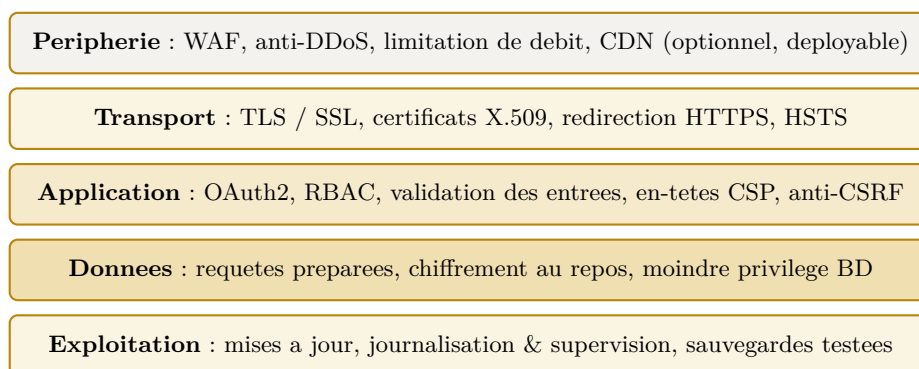


Figure I.1. Defense en profondeur : cinq couches de protection projetees sur l'architecture de Zümm.

I.2.1 Peripherie et transport

En bordure, un **pare-feu applicatif** (WAF, par exemple ModSecurity) filtre les requetes malveillantes et une **limitation de debit** amortit les pics et les tentatives d'abus. Le chiffrement du **transport** est deja exige : TLS / SSL, certificats **X.509**, redirection forcee vers HTTPS et en-tete **Strict-Transport-Security** (HSTS) pour interdire tout repli en clair.

I.2.2 Application

La couche applicative combine l'**authentification** deleguee (OAuth Google, avec double facteur porte par le fournisseur), le **controle d'accès** RBAC au moindre privilege (annexe G), la **validation des entrees**, la **protection anti-CSRF** et les **en-tetes de securite** (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options). La gestion de session s'appuie sur des cookies **HttpOnly** et **Secure** deja specifiques au chapitre 7.

I.2.3 Donnees et exploitation

L'accès aux donnees passe exclusivement par des **requetes preparees** (anti-injection), un compte de base au **moindre privilege** et un **chiffrement au repos** des donnees sensibles. En exploitation, les **mises a jour** regulieres du serveur et des dependances, la **journalisation** et la **supervision**, ainsi que des **sauvegardes chiffrées dont la restauration est testée**, ferment la boucle.

I.3 Grille de durcissement

Le tableau recense les bonnes pratiques de durcissement d'un site web et precise, pour chacune, son **statut** dans Zümm : deja exigee par le cahier des charges ou ajoutée par la presente annexe.

Pratique	Mise en oeuvre dans Zümm	Statut
HTTPS / TLS partout	Certificats X.509, redirection HTTPS, HSTS.	Deja exige
Authentification forte	OAuth Google + double facteur du fournisseur.	Deja exige
Controle d'accès (RBAC)	Roles et permissions au moindre privilege.	Deja exige
Requetes preparees	Acces JDBC exclusivement parametre.	Deja exige
Pare-feu applicatif (WAF)	Filtrage des requetes en peripherie (ModSecurity).	Ajoute
Limitation de debit / anti-DDoS	Amortissement des pics et des abus.	Ajoute
En-tetes de securite	CSP, X-Frame-Options, X-Content-Type-Options.	Ajoute
Protection anti-CSRF	Jeton par session, cookies SameSite.	Ajoute
Mises a jour & correctifs	Serveur d'applications et dependances a jour.	Ajoute
Journalisation & supervision	Traces d'accès et d'erreurs, alertes.	Ajoute
Sauvegardes chiffrees	Sauvegarde reguliere + restauration testee.	Ajoute
Analyse anti-maliciel	Scan periodique des depots et des televersements.	Ajoute

I.4 Robustesse : tests de charge et de fiabilite (k6)

La **disponibilite** etant une propriete de securite, le systeme doit resister a la charge nominale et aux **pics**. On retient **k6**, outil de test de charge **libre**, scriptable, qui simule des *utilisateurs virtuels* (VUs) et verifie des **seuils** objectifs (*thresholds*).

I.4.1 Types de tests

Type	Objectif
Charge (<i>load</i>)	Verifier le comportement a la charge attendue (utilisateurs et mesures nominaux).
Stress	Trouver le point de rupture en augmentant la charge au-dela du nominal.
Pic (<i>spike</i>)	Simuler une rafale brutale, par exemple une remontee massive de mesures capteurs.
Endurance (<i>soak</i>)	Detecter les fuites memoire et la degradation sur une longue duree.

I.4.2 Scenarios propres a Zümm

- **Ingestion en rafale** : POST /api/mesures par lots (volet capteurs, §4.5), pour valider la file asynchrone et l'idempotence.
- **Tableaux de bord** : consultation concurrente des tableaux de bord production et alertes (agregations, graphiques).
- **Service web API** : appels repetes a `getZummHoneyActualQuantity` par des applications tierces.

I.4.3 Seuils et verdict

Le test echoue si les seuils objectifs ne sont pas tenus, ce qui designe un goulot d'étranglement a corriger (taille du pool, index, cache, mise a l'échelle). A titre illustratif :

seuils (thresholds) :

latence p95 des requetes	< 500 ms
taux d'erreurs HTTP	< 1 %
ingestion /api/mesures	debit nominal soutenu sans perte

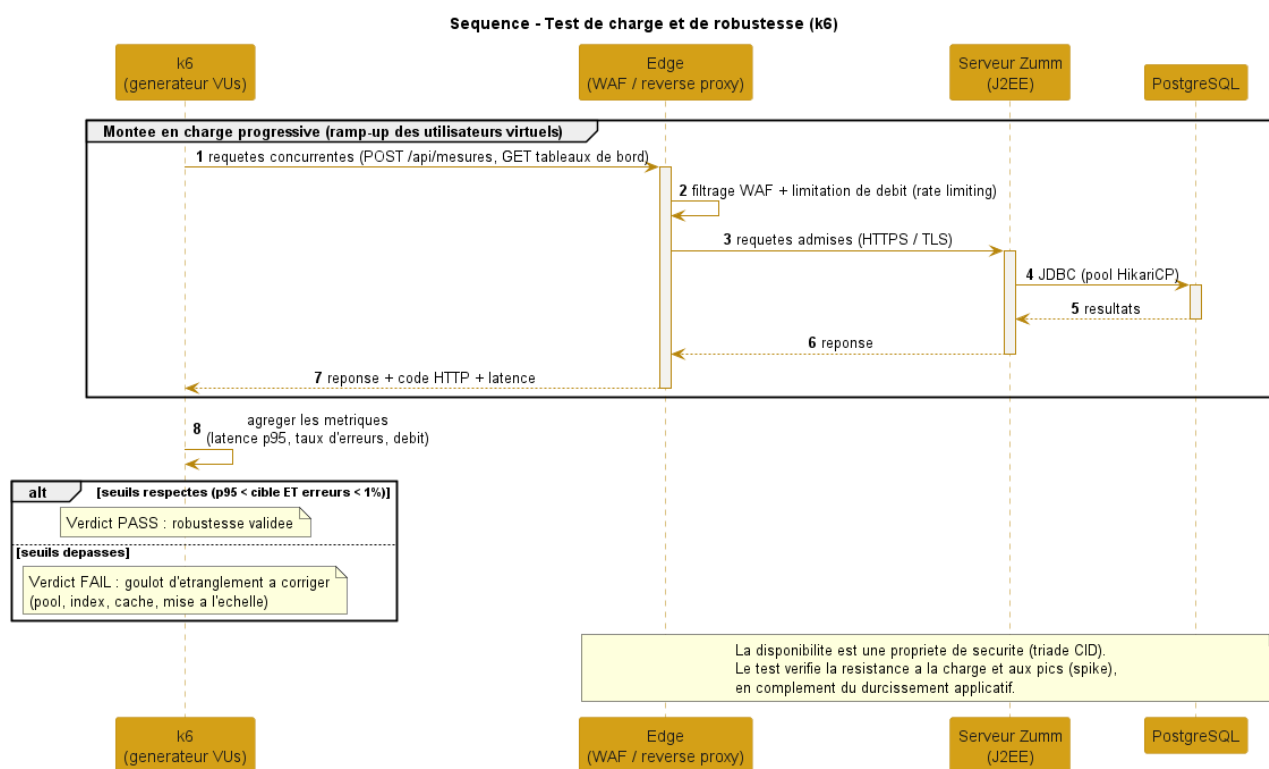


Figure I.2. Test de charge k6 : montee en charge progressive, mesure de la latence et du taux d'erreurs, verdict au regard des seuils.

I.4.4 Complement au plan de tests

Les cas suivants prolongent le plan de tests du chapitre 11 (TC01–TC16) sur le volet securite et robustesse.

ID	Objet	Resultat attendu
TC17	Test de charge (k6)	Seuils p95 et taux d'erreurs respectes a la charge nominale.
TC18	Test de pic (spike)	Le systeme absorbe une rafale sans perte ni erreur durable.

ID	Objet	Resultat attendu
TC19	Injection SQL / XSS	Entrees malveillantes rejetees ou neutralisees (echappement).
TC20	Sauvegarde / restauration	Restauration complete verifiee a partir d'une sauvegarde chiffree.

I.5 Checklist de pre-deploiement (go-live)

Avant toute mise en production, les points ci-dessous sont verifiés. Cette checklist synthetise les sections precedentes en une revue operationnelle, domaine par domaine.

Domaine	Points a verifier avant mise en production
Secrets & configuration	Aucun secret en clair dans le depot ; ConfigZumm.ini , cles et certificats hors versionnement, stockes de facon protegee.
Transport	HTTPS force, redirection du trafic clair, en-tete HSTS actif, certificats X.509 valides et non expires.
Authentification & acces	OAuth Google operationnel, matrice RBAC verifiee, comptes et acces par default desactives.
Application	En-tetes de securite (CSP, X-Frame-Options), protection anti-CSRF, validation des entrees, messages d'erreur sans fuite d'information.
Donnees	Requetes preparees partout, chiffrement au repos des donnees sensibles, sauvegarde recente et restauration testee .
Dependances	Serveur d'applications et bibliotheques a jour, scan de vulnerabilites des dependances effectue.
Robustesse	Test de charge k6 au vert (seuils p95 et taux d'erreurs), test de pic passe, plan de montee en charge et de retour arriere (rollback) defini.
Observabilite	Journalisation des acces et des erreurs, supervision et alertes en place, horodatage coherent.
Conformite	RGPD respecte (minimisation, duree de retention, droits), mentions et registre a jour (annexe G).

Regle de mise en production : un point non tenu est **bloquant** jusqu'a correction. La checklist est rejouee a chaque livraison majeure, dans l'esprit de l'amelioration continue et sous **contrôle humain**.

I.6 Conformite et coherence avec les exigences

Conclusion : l'annexe consolide une securite **par couches** (peripherie, transport, application, donnees, exploitation) et ajoute la **robustesse mesuree** par des tests de charge. Elle prolonge sans les contredire les exigences deja posees (X.509 / SSL, OAuth, RBAC, RGPD) et respecte l'**autodéploiement** : chaque brique retenue (WAF ModSecurity, certificats Let's Encrypt, k6)

est **libre et gratuite**. La securite reste, comme les alertes, une demarche continue placee sous **controle humain**.